

AI Agent Canvas

Multi-Agent Enterprise Copilot Framework

Build intelligent AI copilots with .NET and CopilotKit.
Orchestrate specialized agents that reason, plan, and act
through a shared tool registry.

Complete Reference Guide

Table of Contents

AI Agents

- What Are Ai Agents
- Microsoft Agent Framework
- Tools And Skills
- Model Context Protocol
- Context Providers
- Ag Ui Protocol

Use Cases

- Overview
- Financial Services
- Healthcare
- Legal
- E Commerce
- It Operations

User Guide

- Getting Started
- Configuration
- Chat Interface
- Personas
- Skills And Tools
- Scheduling
- Workflows
- Security

Developer Guide

- Architecture Overview
- Project Structure
- Core Framework
- Agent Data
- Skills And Mcp
- Security

Custom Agents
Custom Mcp Connections

AI Agents

Key concepts behind AI agents, the Microsoft Agent Framework, tools, MCP, context providers, and the AG-UI streaming protocol.

What Are AI Agents

AI agents represent a fundamental shift from traditional chatbots and simple LLM wrappers. Where a chatbot follows scripted flows and an LLM wrapper sends a prompt and returns a response, an AI agent **autonomously reasons, plans, and takes action** to accomplish goals.

Chatbots vs LLM Wrappers vs AI Agents

Capability	Traditional Chatbot	LLM Wrapper	AI Agent
Natural language understanding	Rule-based / intent matching	Full LLM capability	Full LLM capability
Decision making	Scripted decision trees	Single inference call	Multi-step reasoning
Tool use	None or hardcoded	None	Dynamic tool selection
Memory	Session state only	Stateless	Persistent context
Autonomy	None	None	Goal-directed behavior

A **traditional chatbot** matches user input against predefined intents and follows scripted flows. It cannot handle novel requests.

An **LLM wrapper** sends user input to a language model and returns the response. It gains natural language understanding but remains a single request-response cycle with no ability to take action.

An **AI agent** uses an LLM as its reasoning engine while adding the ability to call tools, maintain context across interactions, and execute multi-step plans autonomously.

Key Characteristics of AI Agents

Autonomy

Agents decide **what to do next** without explicit human instruction for each step. Given a goal like "find the latest sales data and summarize trends," an agent determines which tools to call, in what order, and how to synthesize the results.

Tool Use

Agents interact with external systems through **tools** (also called functions). A tool might query a database, call an API, search documents, or perform calculations. The LLM decides which tool to invoke based on the user's request and the tool's description.

Reasoning

Before acting, agents reason about the problem. Modern LLMs can break down complex requests into sub-tasks, evaluate which approach is most appropriate, and adjust their plan based on intermediate results.

Memory and Context

Agents maintain context across interactions. This includes conversation history, retrieved documents (RAG), user preferences, and domain-specific knowledge injected through context providers.

The Agent Loop

Every AI agent follows a fundamental execution cycle:

```
+-----+
| PERCEIVE |
| Receive input, |
| gather context |
+-----+
|
v
+-----+
| REASON |
| Analyze input, |
| plan next step |
+-----+
|
v
+-----+
| ACT |
| Call a tool or |
| generate response|
+-----+
|
v
+-----+
| OBSERVE |
| Process result, |
| decide if done |
+-----+
|
+-----+
| |
v v
Continue Done
(loop) (respond)
```

1. **Perceive** -- The agent receives user input along with injected context (system prompts, RAG results, entity data, user profile information).
2. **Reason** -- The LLM analyzes the input and available tools, then decides on the next action. This may involve breaking a complex request into steps.
3. **Act** -- The agent either calls a tool (e.g., querying a database via MCP) or generates a text response. Tool calls are executed by the framework, not the LLM itself.
4. **Observe** -- The tool result is fed back to the LLM. If the task is complete, the agent produces a final response. If not, it loops back to the Reason step.

This loop continues until the agent determines the task is complete or reaches a configured limit.

Where AI Agent Canvas Fits

AI Agent Canvas is a **multi-agent enterprise copilot framework** built on three pillars:

- **Microsoft Agent Framework (MAF)** for agent orchestration and the LLM execution loop
- **Azure AI Foundry** as the LLM provider (GPT-4o, GPT-4.1, and other models)
- **CopilotKit** with the AG-UI protocol for real-time streaming to the frontend

The framework is designed around separation of concerns:

```
+-----+
| AiAgentCanvas.Web |
| (Composition Root) |
+-----+
| | |
v v v
+-----+ +-----+ +-----+
| Core | | MCP | | MyAgents |
| (Engine) | | (Data) | | (Custom Logic) |
+-----+ +-----+ +-----+
```

- **Core** handles the agent execution loop, tool registry, context providers, and AG-UI streaming. It never changes per use case.
- **MCP** provides data connections. Each MCP project implements data access for a specific domain.
- **MyAgents** contains custom agent reasoning logic. Agents are pure reasoning -- no HTTP calls, no SDK imports for data. They access data exclusively through tools provided by MCP connections.

This architecture means you can build enterprise copilots by composing **agents** (reasoning), **MCP connections** (data), and **context providers** (behavioral instructions) without modifying the core framework.

Next Steps

- [Microsoft Agent Framework](#) -- How MAF powers the agent execution loop
- [Tools and Skills](#) -- How agents interact with external systems
- [Model Context Protocol](#) -- Connecting agents to data sources
- [Context Providers](#) -- Shaping agent behavior through injected context
- [AG-UI Protocol](#) -- Streaming agent responses to the frontend

Microsoft Agent Framework

Microsoft Agent Framework (MAF) is the orchestration layer that powers AI Agent Canvas. It provides the abstractions for building agents that can reason, call tools, and stream responses -- all while integrating with the broader Microsoft.Extensions.AI ecosystem.

AI Agent Canvas uses MAF version **1.10.0** from the `Microsoft.Agents.AI` namespace.

Core Types

IClient

At the foundation, MAF builds on `IClient` from `Microsoft.Extensions.AI`. This interface represents any LLM provider -- Azure AI Foundry, OpenAI, Ollama, or others. AI Agent Canvas configures it through `AIFoundryClientFactory`:

```
services.AddChatClient(sp =>  
    sp.GetRequiredService<AIFoundryClientFactory>().CreateChatClient());
```

This abstraction means the framework is not locked to a specific LLM provider. Swapping from Azure AI Foundry to another provider requires only changing the `IClient` registration.

ChatClientAgent

`ChatClientAgent` is the primary agent implementation in MAF. It wraps an `IClient` and adds the agent execution loop: building messages, attaching tools, calling the LLM, handling tool calls, and streaming responses.

```
var agent = new ChatClientAgent(chatClient, agentOptions, loggerFactory, serviceProvider);
```

The `ChatClientAgent` constructor takes four parameters:

Parameter	Purpose
<code>IClient</code>	The LLM provider to use for inference
<code>ChatClientAgentOptions</code>	Configuration: name, tools, context providers
<code>ILoggerFactory</code>	Structured logging for the agent pipeline
<code>IServiceProvider</code>	DI container for resolving dependencies

ChatClientAgentOptions

This options class configures how the agent behaves:

```
var agentOptions = new ChatClientAgentOptions  
{  
    Name = "AiAgentCanvas",  
    Description = "A multi-tool AI assistant",  
    ChatOptions = new ChatOptions { Tools = tools },  
    ChatHistoryProvider = chatHistoryProvider,  
    AIContextProviders = contextProviders,
```



```
};
```

Key properties:

- **Name / Description** -- Identify the agent in logs and multi-agent scenarios
- **ChatOptions** -- Wraps ChatOptions from Microsoft.Extensions.AI, including the tool list
- **ChatHistoryProvider** -- Optional provider for loading/saving conversation history
- **AIContextProviders** -- Chain of providers that inject instructions, tools, and context before each LLM call

AI Agent

AI Agent is the abstract base class for all agents in MAF. ChatClientAgent inherits from it. The AI Agent type is what gets registered in DI and consumed by endpoints:

```
services.AddSingleton(sp =>
{
// ... configure ChatClientAgent ...
return (AI Agent)builder.Build(sp);
});
```

AgentSession

Each conversation creates an AgentSession that carries per-request state:

```
var session = await agent.CreateSessionAsync(cancellationToken);
session.StateBag.SetValue("conversationId", threadId);
```

The StateBag is a key-value store for session-scoped data. Context providers and middleware can read from and write to it during the agent loop.

The Agent Execution Model

When the agent receives a request, MAF executes the following pipeline:

```
User Message
|
v
+-----+
| AIContextProviders | <-- Inject system prompt, RAG, persona, guardrails
| (chain) |
+-----+
|
v
+-----+
| Build Messages | <-- Conversation history + user message + context
+-----+
|
v
+-----+
| Attach Tools | <-- Static tools + dynamic tools from registry
+-----+
|
```

```

v
+-----+
| Call LLM | <-- IChatClient.GetStreamingChatCompletionAsync
+-----+
|
+-----+-----+
| |
v v
Text Tool Call
Response +-----+
(stream) | Execute Tool |
+-----+
|
v
+-----+
| Feed Result Back | <-- Loop to "Call LLM" with tool result
+-----+

```

Step 1: Context Injection

Before the LLM is called, MAF iterates through the `AIContextProviders` chain. Each provider can modify the `AIContext` -- adding instructions, appending tools, or injecting retrieved documents. Providers execute in registration order.

Step 2: Message Construction

The framework combines conversation history (from `ChatHistoryProvider`), the current user message, and the system instructions assembled by context providers into a single message list.

Step 3: Tool Attachment

Tools registered in `ChatOptions` and any dynamic tools added by `DynamicToolContextProvider` are attached to the LLM request. The LLM receives the tool schemas (name, description, parameters) so it can decide when to call them.

Step 4: LLM Invocation

MAF calls `IChatClient` with the messages and tools. For streaming, it uses `GetStreamingChatCompletionAsync`, yielding tokens as they arrive.

Step 5: Tool Call Handling

If the LLM returns a tool call instead of (or alongside) text, MAF:

1. Parses the tool name and arguments from the LLM response
2. Finds the matching `AITool` in the registered set
3. Executes the tool with the provided arguments
4. Appends the tool result as a new message
5. Calls the LLM again with the updated message list

This loop continues until the LLM produces a final text response without further tool calls.

Middleware Pipeline

MAF supports middleware that wraps the agent execution. AI Agent Canvas uses this for logging:

```
var builder = agent.AsBuilder();
builder.Use(async (messages, session, runOptions, nextAsync, ct) =>
{
    logger.LogInformation("Agent invoked. Messages={Count}", messages.Count());
    await nextAsync(messages, session, runOptions, ct);
    logger.LogInformation("Agent completed in {ElapsedMs}ms", sw.ElapsedMilliseconds);
});
```

Middleware receives the full pipeline context and calls `nextAsync` to continue execution. This pattern is familiar from ASP.NET Core middleware and enables cross-cutting concerns like:

- **Logging** -- Timing and message counts
- **Metrics** -- Token usage and tool call counts
- **Error handling** -- Catching and recovering from LLM failures
- **Caching** -- Short-circuiting repeated queries

Running the Agent

The agent exposes two execution methods:

- `RunAsync` -- Returns the complete response after all tool calls resolve
- `RunStreamingAsync` -- Yields response chunks as an `IAsyncEnumerable`, enabling SSE streaming

AI Agent Canvas uses `RunStreamingAsync` in the AG-UI endpoint:

```
await foreach (var update in agent.RunStreamingAsync(
    chatMessages, session, cancellationToken: ct))
{
    if (update.Text is { Length: > 0 } text)
    {
        await WriteEvent(context, "text.message.content",
            new { messageId, delta = text });
    }
}
```

Summary

MAF provides the execution backbone for AI Agent Canvas:

Concern	MAF Type
LLM abstraction	<code>IChatClient</code>
Agent orchestration	<code>ChatClientAgent</code> / <code>AIAgent</code>
Configuration	<code>ChatClientAgentOptions</code>
Context injection	<code>AIContextProvider</code> chain

Session state	AgentSession / StateBag
Extensibility	Middleware pipeline via AsBuilder()

The framework handles the complexity of the agent loop -- tool call resolution, message management, streaming -- so that application code focuses on defining tools, context providers, and business logic.

Tools and Skills

Tools are how AI agents interact with the world. Without tools, an agent can only generate text. With tools, it can query databases, call APIs, search documents, manage schedules, and perform calculations. Skills extend this concept by allowing agents to create and manage reusable prompt-based tools at runtime.

What Are AI Tools

In the Microsoft.Extensions.AI ecosystem, a tool is represented by the `AITool` base class. The most common concrete type is `AIFunction`, which wraps a .NET method so the LLM can call it.

The LLM never executes tools directly. Instead, it receives tool **schemas** (name, description, parameter definitions) and decides when to call them. The framework handles execution and feeds results back.

AIFunction and AIFunctionFactory

`AIFunctionFactory.Create` is the primary way to define tools in AI Agent Canvas. It takes a delegate and metadata, producing an `AITool` the framework can register:

```
public IReadOnlyList<AITool> GetTools()
{
    return
    [
        AIFunctionFactory.Create(ConnectMcpServer, "connect_mcp_server",
            "Connect to an MCP server and register its tools"),
        AIFunctionFactory.Create(DisconnectMcpServer, "disconnect_mcp_server",
            "Disconnect from an MCP server and remove its tools"),
        AIFunctionFactory.Create(ListMcpConnections, "list_mcp_connections",
            "List all active MCP server connections"),
    ];
}
```

The `[Description]` attribute on parameters generates the JSON schema that the LLM uses to understand what arguments to provide:

```
[Description("Connect to an MCP server and register its tools")]
private async Task<string> ConnectMcpServer(
    [Description("A unique name for this connection")] string name,
    [Description("The server endpoint URL")] string endpoint,
    [Description("Transport type: 'http' or 'sse'")] string transport,
    CancellationToken ct)
{
    // Implementation
}
```

Good tool descriptions are critical. The LLM relies on them to decide **when** and **how** to call a tool. Vague descriptions lead to incorrect tool selection.

The Tool Call Loop

When the LLM decides to use a tool, the following sequence executes:

```
User: "What MCP servers are connected?"
|
v
+-----+
| LLM Reasons |
| "I should call |
| list_mcp_ |
| connections" |
+-----+-----+
|
v
+-----+
| Framework |
| executes |
| ListMcp...() |
+-----+-----+
|
v
+-----+
| Result: |
| {"count": 2, |
| "connections":.|
| [...] } |
+-----+-----+
|
v
+-----+
| LLM receives |
| result, formats|
| human-readable |
| response |
+-----+
```

Key points about tool execution:

1. **The LLM chooses** -- The framework never forces a tool call. The LLM infers from context and tool descriptions.
2. **The framework executes** -- Tool methods run in the server process, not in the LLM. This is important for security and data access.
3. **Results feed back** -- Tool output becomes a new message in the conversation, and the LLM generates its response with that information.
4. **Multiple rounds** -- The LLM can call multiple tools in sequence, using each result to inform the next step.

DynamicToolRegistry

Not all tools are known at startup. AI Agent Canvas uses `DynamicToolRegistry` to compose tools from multiple sources at runtime:

```
public sealed class DynamicToolRegistry
{
    private readonly ConcurrentDictionary<string, List<AITool>> _toolsBySource = new();
```

```

public void Register(string source, IEnumerable<AITool> tools)
{
    _toolsBySource[source] = tools.ToList();
}

public void Unregister(string source)
{
    _toolsBySource.TryRemove(source, out _);
}

public IReadOnlyList<AITool> GetAllTools()
{
    return _toolsBySource.Values.SelectMany(t => t).ToList();
}
}

```

Tools are grouped by **source** -- a string key that identifies where they came from. For example:

- "mcp:market-data" -- Tools from a connected MCP server
- "skills" -- Tools loaded from skill definitions
- "system" -- Built-in system tools

The `DynamicToolContextProvider` injects these dynamic tools into every LLM call:

```

internal sealed class DynamicToolContextProvider : AIToolContextProvider
{
    private readonly DynamicToolRegistry _registry;

    protected override ValueTask<AIToolContext> ProvideAIToolContextAsync(
        InvokingContext context, CancellationToken cancellationToken)
    {
        var dynamicTools = _registry.GetAllTools();
        if (dynamicTools.Count > 0)
        {
            var existing = context.AIToolContext.Tools?.ToList() ?? [];
            existing.AddRange(dynamicTools);
            context.AIToolContext.Tools = existing;
        }
        return new ValueTask<AIToolContext>(context.AIToolContext);
    }
}

```

This means when an MCP server is connected at runtime, its tools immediately become available to the agent without a restart.

Tool Providers

AI Agent Canvas organizes tools into **tool provider** classes. Each provider owns a specific domain of tools:

Provider	Tools	Purpose
McpConnectionManager	connect_mcp_server, disconnect_mcp_server, list_mcp_connections	Manage MCP data connections

SkillToolProvider	create_skill, list_skills, run_skill, remove_skill	Manage reusable prompt skills
SkillAuthoringToolProvider	author_skill, edit_skill, read_skill, delete_authored_skill	Author skill markdown files
SchedulerToolProvider	Schedule-related tools	Manage scheduled agent tasks
WorkflowToolProvider	Workflow tools	Execute multi-step workflows
SystemToolProvider	System-level utilities	Core system operations

Tool providers implement a `GetTools()` method that returns their tools as `ReadOnlyList<AITool>`. These are registered in DI and collected during agent construction.

Skills: Agent-Managed Tools

Skills are a higher-level abstraction on top of tools. A skill is a **persisted prompt template** that the agent can create, manage, and execute at runtime.

Creating a Skill

When a user asks the agent to "create a skill that summarizes text," the agent calls `create_skill`:

```
private string CreateSkill(string name, string description, string promptTemplate)
{
    var record = new SkillRecord
    {
        Id = id,
        Name = normalizedName,
        Description = description,
        PromptTemplate = promptTemplate, // e.g., "Summarize: {input}"
    };
    _store.SaveSkill(record);
}
```

Running a Skill

When the skill is invoked via `run_skill`, the framework substitutes the `{input}` placeholder and sends the expanded prompt to the agent:

```
private async Task<string> RunSkill(string name, string input, CancellationToken ct)
{
    var skill = _store.GetSkill(name);
    var prompt = skill.PromptTemplate.Replace("{input}", input);

    var messages = new List<ChatMessage>
    {
        new(ChatRole.User, prompt),
    };

    var response = await _agent.RunAsync(messages, cancellationToken: ct);
    return JsonSerializer.Serialize(new { skill = name, result = response.Text });
}
```

Skill Authoring

Beyond runtime skill creation, `SkillAuthoringToolProvider` allows the agent to write skill definitions as **markdown files** with YAML frontmatter:

```
---
name: summarize
description: Summarize text into key points
tags: text, summary
---

Summarize the following text into 3-5 key bullet points:

{input}
```

This persistence format makes skills portable, version-controllable, and human-readable.

Tool Design Guidelines

When building tools for AI Agent Canvas:

1. **Descriptive names** -- Use verb-noun format: `connect_mcp_server`, not `mcp` or `connect`.
2. **Clear descriptions** -- Both the tool and each parameter need descriptions the LLM can reason about.
3. **Return structured data** -- Return JSON strings so the LLM can parse and present results cleanly.
4. **Handle errors gracefully** -- Return error information as data rather than throwing exceptions.
5. **Keep tools focused** -- One tool per action. Avoid multi-purpose tools with mode parameters.
6. **Use CancellationToken** -- Long-running tools should accept and respect cancellation tokens.

Summary

The tools and skills system in AI Agent Canvas provides a layered approach:

- **AITool / AIFunction** -- The foundation: .NET methods the LLM can call
- **Tool Providers** -- Organized groups of related tools
- **DynamicToolRegistry** -- Runtime composition of tools from multiple sources
- **Skills** -- Persisted prompt templates the agent manages as reusable capabilities

Model Context Protocol

Model Context Protocol (MCP) is an open standard for connecting LLMs to external data sources, tools, and services. Rather than building custom integrations for every data source, MCP provides a **universal protocol** that any LLM-powered application can use to discover and call tools exposed by any MCP-compliant server.

Why MCP Matters

Before MCP, connecting an AI agent to a data source required:

1. Writing custom API client code
2. Defining tool schemas manually
3. Handling authentication, serialization, and error handling per integration
4. Rebuilding when the data source API changes

MCP standardizes this into a single protocol:

```
+-----+ MCP Protocol +-----+
| Agent | <-----> | MCP Server |
| (Client) | discover tools, call them, | (Data |
| | receive results | Source) |
+-----+ +-----+
```

An MCP server exposes a set of tools with standardized schemas. An MCP client (the agent) discovers those tools at runtime, and the LLM can call them like any local tool. The agent does not need to know the implementation details of the data source.

MCP in AI Agent Canvas

AI Agent Canvas implements MCP through two main components:

- `McpConnectionManager` -- Manages the lifecycle of MCP connections (connect, disconnect, list)
- `DynamicToolRegistry` -- Registers MCP tools so the agent can use them alongside local tools

McpClient and HttpClientTransport

The framework uses the `ModelContextProtocol.Client` library to connect to MCP servers:

```
IClientTransport clientTransport = transport.ToLowerInvariant() switch
{
    "http" or "sse" => new HttpClientTransport(
        new HttpClientTransportOptions
        {
            Endpoint = new Uri(endpoint)
        },
    ),
    _ => throw new ArgumentException($"Unsupported transport: {transport}"),
};
```

```
var client = await McpClient.CreateAsync(clientTransport, cancellation_token: ct);
```

`HttpClientTransport` handles the HTTP-based communication with MCP servers, supporting both standard HTTP request/response and SSE (Server-Sent Events) streaming for real-time tool execution.

Tool Discovery

Once connected, the client discovers available tools:

```
var mcpTools = await client.ListToolsAsync(cancellation_token: ct);  
var aiTools = mcpTools.Cast<AITool>().ToList();
```

MCP tools implement `AITool`, so they integrate seamlessly with the `Microsoft.Extensions.AI` tool system. The agent sees no difference between a local tool and an MCP tool -- both appear in the same tool list with the same schema format.

Dynamic MCP Connections

A key design principle of AI Agent Canvas is that MCP connections are **runtime-dynamic**. The agent itself can connect to and disconnect from MCP servers during a conversation.

Connecting

The agent exposes a `connect_mcp_server` tool that users can invoke through natural language:

```
User: "Connect to the market data server at https://mcp.example.com/market"
```

The agent calls `connect_mcp_server`, which:

1. Creates an `HttpClientTransport` with the specified endpoint
2. Establishes the MCP connection via `McpClient.CreateAsync`
3. Discovers available tools via `ListToolsAsync`
4. Registers the tools in `DynamicToolRegistry` under the source key `mcp:{name}`
5. Returns the list of discovered tools to the LLM

```
_connections[name] = connection;  
_registry.Register($"mcp:{name}", aiTools);
```

From this point forward, the agent can call any tool from the connected MCP server.

Disconnecting

When a connection is no longer needed:

```
private async Task<string> DisconnectMcpServer(string name, CancellationToken ct)  
{  
    if (!_connections.TryRemove(name, out var connection))  
        return JsonSerializer.Serialize(new { error = $"Connection '{name}' not found" });  
  
    _registry.Unregister($"mcp:{name}");  
}
```

```

if (connection.Client is not null)
    await connection.Client.DisposeAsync();

return JsonSerializer.Serialize(new { status = "disconnected", name });
}

```

Disconnecting removes the tools from the registry and disposes the MCP client. The agent immediately loses access to those tools.

Listing Connections

The `list_mcp_connections` tool shows all active connections and their tool counts, giving the agent (and user) visibility into what data sources are available.

MCP Tools vs Local Tools

AI Agent Canvas uses both MCP tools and local tools. Understanding the distinction helps when designing the system:

Aspect	Local Tools	MCP Tools
Definition	C# methods via <code>AIFunctionFactory</code>	Defined by remote MCP server
Registration	At startup in <code>Program.cs</code>	At runtime via <code>McpConnectionManager</code>
Execution	In-process	Remote HTTP call to MCP server
Latency	Minimal	Network-dependent
Availability	Always available	Only while connected
Schema source	[Description] attributes	MCP server's tool listing

From the LLM's perspective, both types are identical. The framework abstracts the execution mechanism, so the agent reasons about tools purely based on their names and descriptions.

Tool Composition Example

A typical AI Agent Canvas deployment might have:

```

Agent Tool List
|
+-- Local Tools (always available)
| +-- create_skill
| +-- run_skill
| +-- connect_mcp_server
| +-- list_mcp_connections
|
+-- MCP: market-data (connected at runtime)
| +-- get_stock_price
| +-- get_market_summary
| +-- search_tickers
|
+-- MCP: crm (connected at runtime)
+-- search_contacts

```

```
+++ get_deal_pipeline
+++ update_opportunity
```

The `DynamicToolRegistry` merges all sources into a flat list. The LLM sees all tools equally and picks the right one based on the user's request.

Security Considerations

Dynamic MCP connections introduce security concerns that AI Agent Canvas addresses through its governance layer.

SSRF Protection

The `GovernedMcpGateway` evaluates every MCP tool call against a governance policy before execution:

```
public sealed class GovernedMcpGateway
{
    private readonly McpGateway _gateway;

    public McpGatewayDecision Evaluate(
        string agentId, string toolName, string? payload = null)
    {
        var request = new McpGatewayRequest
        {
            AgentId = agentId,
            ToolName = toolName,
            Payload = payload ?? "",
        };

        var decision = _gateway.ProcessRequest(request);

        if (!decision.Allowed)
        {
            _logger.LogWarning(
                "[GOVERNANCE:MCP] Blocked tool={Tool} agent={Agent} status={Status}",
                toolName, agentId, decision.Status);
        }

        return decision;
    }
}
```

This prevents:

- **SSRF attacks** -- An attacker cannot trick the agent into connecting to internal network endpoints
- **Unauthorized tool use** -- Policies can restrict which agents can call which MCP tools
- **Data exfiltration** -- Outbound tool calls are audited and can be blocked based on payload inspection

Best Practices

When deploying MCP in production:

1. **Allowlist endpoints** -- Only permit connections to known, trusted MCP server URLs
2. **Use HTTPS** -- Always require TLS for MCP transport
3. **Audit connections** -- Log all connect/disconnect events and tool invocations
4. **Scope permissions** -- Use the governance policy to restrict tool access per agent
5. **Set timeouts** -- Configure connection and request timeouts to prevent hanging connections
6. **Dispose properly** -- The `McpConnectionManager` implements `IAsyncDisposable` to clean up connections on shutdown

Summary

MCP transforms how AI agents access data. Instead of hardcoded integrations, agents discover and use tools from any MCP-compliant server at runtime. AI Agent Canvas's implementation makes this dynamic and governed:

- `McpConnectionManager` handles the connection lifecycle
- `DynamicToolRegistry` merges MCP tools with local tools transparently
- `GovernedMcpGateway` enforces security policies on MCP tool calls
- The agent can connect/disconnect MCP servers during a conversation, adapting to the user's needs

Context Providers

Context providers are the mechanism by which AI Agent Canvas shapes agent behavior **without modifying the agent itself**. They inject instructions, domain knowledge, guardrails, and user-specific information into the agent's context before every LLM call.

What Are Context Providers

A context provider is a subclass of `AIContextProvider` from the Microsoft Agent Framework (`Microsoft.Agents.AI`). Each provider implements a single method that can modify the `AIContext` -- adding to the system instructions, attaching tools, or injecting retrieved documents:

```
public abstract class AIContextProvider
{
    protected abstract ValueTask<AIContext> ProvideAIContextAsync(
        InvokingContext context,
        CancellationToken cancellationToken);
}
```

The `InvokingContext` gives the provider access to:

- `AIContext` -- The current context being built (instructions, tools, messages)
- **Session state** -- Via the agent session's `StateBag`

Providers return the modified `AIContext`, which flows to the next provider in the chain.

The Context Injection Chain

Context providers execute in **registration order** before every LLM call. Each provider layers its contribution on top of what previous providers set:

```
LLM Call Request
|
v
+-----+
| SystemPromptProvider | --> Sets base system instructions
+-----+
|
v
+-----+
| PersonaContextProvider | --> Appends persona-specific instructions
+-----+
|
v
+-----+
| UserProfileContext... | --> Appends user preferences and profile data
+-----+
|
v
+-----+
| EntityContextProvider | --> Appends entity/domain knowledge index
+-----+
|
v
```

```

+-----+
| GuardrailContext... | --> Appends behavioral rules and constraints
+-----+
|
v
+-----+
| RagContextProvider | --> Appends retrieved documents from vector search
+-----+
|
v
+-----+
| GovernanceContext... | --> Scans for prompt injection, audits
+-----+
|
v
+-----+
| DynamicToolContext... | --> Attaches runtime tools from DynamicToolRegistry
+-----+
|
v
Final AIContext sent to LLM

```

This layered approach means each provider is **independent and composable**. Adding a new provider does not require changes to existing ones.

Built-in Context Providers

SystemPromptProvider

The foundation of the context chain. Sets the base system instructions if none are already set:

```

internal sealed class SystemPromptProvider : AIContextProvider
{
    private readonly string _systemPrompt;

    protected override ValueTask<AIContext> ProvideAIContextAsync(
        InvokingContext context, CancellationToken cancellationToken)
    {
        if (string.IsNullOrEmpty(context.AIContext.Instructions))
            context.AIContext.Instructions = _systemPrompt;
        return new ValueTask<AIContext>(context.AIContext);
    }
}

```

The default prompt is configurable via `AiAgentCanvasOptions.SystemPrompt` or defaults to a generic helpful assistant prompt.

PersonaContextProvider

Appends persona-specific instructions from the `PersonaStore`. Personas define **how** the agent should behave -- tone, expertise level, domain focus:

```

internal sealed class PersonaContextProvider : AIContextProvider
{
    private readonly PersonaStore _store;
}

```



```
protected override ValueTask<AIContext> ProvideAIContextAsync(
    InvokingContext context, CancellationToken cancellationToken)
{
    var activeInstructions = _store.GetActiveInstructions();
    if (!string.IsNullOrEmpty(activeInstructions))
    {
        context.AIContext.Instructions =
            (context.AIContext.Instructions ?? "") + "\n" + activeInstructions;
    }
    return new ValueTask<AIContext>(context.AIContext);
}
}
```

Example persona instructions might be: "You are a financial analyst specializing in equity markets. Use precise numbers and cite data sources. Respond in a professional, concise tone."

UserProfileContextProvider

Injects user-specific preferences and information:

```
internal sealed class UserProfileContextProvider : AIContextProvider
{
    private readonly UserProfileStore _store;

    protected override ValueTask<AIContext> ProvideAIContextAsync(
        InvokingContext context, CancellationToken cancellationToken)
    {
        var profileContext = _store.LoadActiveProfileContext();
        if (!string.IsNullOrEmpty(profileContext))
        {
            context.AIContext.Instructions =
                (context.AIContext.Instructions ?? "") + profileContext;
        }
        return new ValueTask<AIContext>(context.AIContext);
    }
}
```

This enables personalization without the user repeating preferences. The profile might include role, team, preferred output format, or domain-specific terminology.

EntityContextProvider

Appends a domain knowledge index from the EntityStore. Entities represent key concepts, terms, or reference data the agent should know about:

```
internal sealed class EntityContextProvider : AIContextProvider
{
    private readonly EntityStore _store;

    protected override ValueTask<AIContext> ProvideAIContextAsync(
        InvokingContext context, CancellationToken cancellationToken)
    {
        var index = _store.LoadEntityIndex();
        if (!string.IsNullOrEmpty(index))
        {
            context.AIContext.Instructions =
                (context.AIContext.Instructions ?? "") + index;
        }
    }
}
```

```
return new ValueTask<AIContext>(context.AIContext);
}
}
```

For example, an entity index might define company-specific product names, internal acronyms, or organizational structure that the LLM would not know from training data.

GuardrailContextProvider

Appends behavioral constraints and rules:

```
internal sealed class GuardrailContextProvider : AIContextProvider
{
    private readonly GuardrailStore _store;

    protected override ValueTask<AIContext> ProvideAIContextAsync(
        InvokingContext context, CancellationToken cancellationToken)
    {
        var rules = _store.LoadActiveRules();
        if (!string.IsNullOrEmpty(rules))
        {
            context.AIContext.Instructions =
                (context.AIContext.Instructions ?? "") + rules;
        }
        return new ValueTask<AIContext>(context.AIContext);
    }
}
```

Guardrail rules might include: "Never provide specific investment advice. Always include disclaimers when discussing financial data. Do not share PII across user sessions."

RagContextProvider

Retrieves relevant documents from a vector store and appends them to the instructions:

```
public sealed class RagContextProvider : AIContextProvider
{
    protected override async ValueTask<AIContext> ProvideAIContextAsync(
        InvokingContext context, CancellationToken cancellationToken)
    {
        var lastUserMessage = context.AIContext.Messages?
            .LastOrDefault(m => m.Role == ChatRole.User)?.Text;

        if (string.IsNullOrEmptyOrWhiteSpace(lastUserMessage))
            return context.AIContext;

        var queryEmbedding = await _embeddingGenerator
            .GenerateVectorAsync(lastUserMessage, cancellationToken: cancellationToken);

        var results = new List<string>();
        await foreach (var result in _collection
            .SearchAsync(queryEmbedding, _topK, cancellationToken: cancellationToken))
        {
            results.Add(result.Record.Text);
        }

        var ragContext = string.Join("\n\n---\n\n", results);
        context.AIContext.Instructions +=
```

```

$"\n\nRelevant context from documents:\n{ragContext}";
return context.AIContext;
}
}

```

RAG differs from other providers in that it is **query-dependent** -- it reads the last user message, generates an embedding, and searches a vector store for relevant content.

GovernanceContextProvider

Scans the assembled instructions for prompt injection attempts:

```

public sealed class GovernanceContextProvider : AIContextProvider
{
    protected override ValueTask<AIContext> ProvideAIContextAsync(
        InvokingContext context, CancellationToken cancellation)
    {
        var result = _kernel.InjectionDetector.Detect(context.AIContext.Instructions);
        if (result.IsInjection)
        {
            _logger.LogWarning(
                "[GOVERNANCE] Prompt injection detected: {Type}",
                result.InjectionType);

            _kernel.AuditEmitter.Emit(
                GovernanceEventType.PolicyViolation, ...);
        }
        return new ValueTask<AIContext>(context.AIContext);
    }
}

```

This provider typically runs **last** in the chain so it can scan the fully assembled instructions.

DynamicToolContextProvider

Injects runtime tools from the `DynamicToolRegistry`. Unlike other providers that modify instructions, this one adds **tools**:

```

internal sealed class DynamicToolContextProvider : AIContextProvider
{
    protected override ValueTask<AIContext> ProvideAIContextAsync(
        InvokingContext context, CancellationToken cancellation)
    {
        var dynamicTools = _registry.GetAllTools();
        if (dynamicTools.Count > 0)
        {
            var existing = context.AIContext.Tools?.ToList() ?? [];
            existing.AddRange(dynamicTools);
            context.AIContext.Tools = existing;
        }
        return new ValueTask<AIContext>(context.AIContext);
    }
}

```

Creating Custom Context Providers

To add a custom context provider, subclass `AIContextProvider` and register it in DI:

```
public sealed class CompanyKnowledgeProvider : AIContextProvider
{
    private readonly IKnowledgeBase _kb;

    public CompanyKnowledgeProvider(IKnowledgeBase kb) => _kb = kb;

    protected override async ValueTask<AIContext> ProvideAIContextAsync(
        InvokingContext context, CancellationToken cancellationToken)
    {
        var knowledge = await _kb.GetRelevantFacts(cancellationToken);
        if (!string.IsNullOrEmpty(knowledge))
        {
            context.AIContext.Instructions =
                (context.AIContext.Instructions ?? "") +
                $"\n\nCompany knowledge:\n{knowledge}";
        }
        return context.AIContext;
    }
}
```

Register it in `Program.cs`:

```
services.AddSingleton<AIContextProvider, CompanyKnowledgeProvider>();
```

The provider will execute as part of the chain, in the order it was registered.

Design Guidelines

1. **Single responsibility** -- Each provider should handle one concern (persona, guardrails, RAG, etc.)
2. **Append, don't replace** -- Append to existing instructions rather than overwriting them, unless you are the system prompt provider
3. **Fail gracefully** -- If a provider's data source is unavailable, return the context unchanged rather than throwing
4. **Keep instructions concise** -- Every token in the instructions counts against the context window and adds cost
5. **Order matters** -- Register providers in logical order: base prompt first, then layered context, then guardrails, then governance scanning last

AG-UI Protocol

AG-UI (Agent Graphical UI) is a protocol for streaming AI agent responses to frontend applications in real time. AI Agent Canvas uses AG-UI to connect the .NET backend (powered by Microsoft Agent Framework) to the CopilotKit frontend over Server-Sent Events (SSE).

What Is AG-UI

Traditional REST APIs return a complete response in a single payload. For AI agents that may take seconds to generate a response, this creates a poor user experience -- the user stares at a loading spinner until the entire response is ready.

AG-UI solves this by streaming **events** as the agent processes. The user sees text appear token by token, knows when the agent starts and finishes, and gets a responsive conversational experience.

The protocol defines a set of event types that represent the lifecycle of an agent response:

```
Client POST /api/copilotkit
|
v
Server opens SSE stream
|
+-- event: run.started
+-- event: text.message.start
+-- event: text.message.content (repeated, one per token chunk)
+-- event: text.message.content
+-- event: text.message.content
+-- event: text.message.end
+-- event: run.finished
|
v
SSE stream closes
```

The /api/copilotkit Endpoint

AI Agent Canvas exposes a single SSE endpoint that CopilotKit connects to:

```
public static class AgUiEndpoint
{
    public static void MapAgUiEndpoints(this WebApplication app)
    {
        app.MapPost("/api/copilotkit", HandleCopilotKitRequest);
    }
}
```

This endpoint is mapped in the application startup via `app.UseAiAgentCanvas()`, which internally calls `app.MapAgUiEndpoints()`.

Request Format

The frontend sends a POST request with a JSON body containing the conversation messages and an optional thread ID:

```
{
  "threadId": "conv-abc-123",
  "messages": [
    {
      "role": "system",
      "content": "You are a helpful assistant."
    },
    {
      "role": "user",
      "content": "What is the current market summary?"
    }
  ]
}
```

The endpoint extracts messages and initializes the agent session:

```
var body = await JsonSerializer.DeserializeAsync<JsonElement>(context.Request.Body);
var messages = body.GetProperty("messages");
var threadId = body.TryGetProperty("threadId", out var tid)
? tid.GetString() ?? Guid.NewGuid().ToString()
: Guid.NewGuid().ToString();
var runId = Guid.NewGuid().ToString();
```

Each request gets a unique runId. The threadId persists across messages in the same conversation, enabling continuity.

SSE Response Format

The response uses the text/event-stream content type with standard SSE formatting:

```
context.Response.ContentType = "text/event-stream";
context.Response.Headers.CacheControl = "no-cache";
context.Response.Headers.Connection = "keep-alive";
```

Each event is written as:

```
event: <event-type>
data: <json-payload>
```

Note the double newline after the data line -- this is the SSE event delimiter.

Event Types

run.started

Signals that the agent has begun processing the request:

```
event: run.started
data: {"threadId": "conv-abc-123", "runId": "run-xyz-789"}
```

The frontend uses this to show a processing indicator and track the run.

text.message.start

Marks the beginning of a new assistant message:

```
event: text.message.start
data: {"messageId":"msg-001","role":"assistant","agentName":"AiAgentCanvas"}
```

CopilotKit creates a new message bubble in the UI when it receives this event.

text.message.content

Delivers a chunk of the response text. This event is sent repeatedly as tokens stream from the LLM:

```
event: text.message.content
data: {"messageId":"msg-001","delta":"The current"}

event: text.message.content
data: {"messageId":"msg-001","delta":" market shows"}

event: text.message.content
data: {"messageId":"msg-001","delta":" strong momentum..."}
```

The delta field contains the incremental text. The frontend appends each delta to build the complete message progressively.

text.message.end

Signals that the message is complete:

```
event: text.message.end
data: {"messageId":"msg-001"}
```

run.finished

Signals that the entire agent run is complete:

```
event: run.finished
data: {"threadId":"conv-abc-123","runId":"run-xyz-789"}
```

The frontend uses this to stop the processing indicator and re-enable the input field.

Streaming Implementation

The backend iterates over the agent's streaming output and emits SSE events for each chunk:

```
await foreach (var update in agent.RunStreamingAsync(
    chatMessages, session, cancellationToken: context.RequestAborted))
{
    if (update.Text is { Length: > 0 } text)
    {
        await WriteEvent(context, "text.message.content",
            new { messageId, delta = text });
    }
}
```

The WriteEvent helper formats and flushes each event:

```
private static async Task WriteEvent(
    HttpContext context, string eventType, object data)
{
    var json = JsonSerializer.Serialize(data, JsonOptions);
```

```
await context.Response.WriteAsync(
  $"event: {eventType}\\ndata: {json}\\n\\n",
  context.RequestAborted);
await context.Response.Body.FlushAsync(context.RequestAborted);
}
```

The `FlushAsync` call is critical -- without it, the response may be buffered and the client would not receive events in real time.

How CopilotKit Consumes the Stream

On the frontend, CopilotKit handles the SSE stream automatically. The Next.js application configures CopilotKit with the backend URL:

```
<CopilotKit runtimeUrl="/api/copilotkit">
  <CopilotChat />
</CopilotKit>
```

CopilotKit:

1. Sends POST requests to `/api/copilotkit` when the user submits a message
2. Opens an SSE connection on the response
3. Parses incoming events by type
4. Updates the chat UI progressively as `text.message.content` events arrive
5. Manages conversation state across multiple request/response cycles

Why SSE Over WebSockets

AI Agent Canvas uses SSE rather than WebSockets for several reasons:

Consideration	SSE	WebSockets
Direction	Server-to-client (unidirectional)	Bidirectional
HTTP compatibility	Standard HTTP, works with all proxies/CDNs	Requires protocol upgrade
Reconnection	Built-in automatic reconnect	Must implement manually
Complexity	Simple text protocol	Binary framing, connection management
Load balancers	Works with standard HTTP load balancers	Requires sticky sessions or special config
Firewall friendliness	Standard HTTPS port, no special rules	May be blocked by corporate firewalls

For the agent use case, communication is fundamentally **request-response with streaming**: the client sends a message (POST), and the server streams back the response. There is no need for the server to push unsolicited messages or for the client to send data mid-stream.

SSE fits this pattern naturally:

- The POST request carries the user message
- The SSE response carries the streaming agent response
- HTTP infrastructure (proxies, CDNs, load balancers) works without modification
- Automatic reconnection handles network interruptions

WebSockets would add complexity (connection management, heartbeats, reconnection logic) without providing benefits for this use case.

Cancellation

The endpoint respects the HTTP request's cancellation token:

```
cancellationToken: context.RequestAborted
```

If the client closes the connection (user navigates away, network drops), the `RequestAborted` token fires, canceling the agent's LLM call and stopping the stream. This prevents wasted compute on responses nobody will read.

Summary

The AG-UI protocol provides a clean boundary between the .NET agent backend and the CopilotKit frontend:

- **POST /api/copilotkit** -- Single endpoint for all agent interactions
- **SSE events** -- `run.started`, `text.message.start`, `text.message.content`, `text.message.end`, `run.finished`
- **Progressive rendering** -- Users see responses as they are generated, token by token
- **Standard HTTP** -- No special infrastructure required, works with existing proxies and load balancers
- **Cancellation** -- Clean shutdown when clients disconnect

Use Cases

Real-world scenarios showing how AI Agent Canvas applies to financial services, healthcare, legal, e-commerce, and IT operations.

Use Cases

AI Agent Canvas is a multi-agent enterprise copilot framework that adapts to any industry. The following scenarios demonstrate how organizations use tool-calling agents, MCP data connections, scheduled tasks, skill authoring, personas, guardrails, entity management, user profiles, and security governance to solve real operational challenges.

Each use case presents a hypothetical company, a specific persona, the problem they face, and how AI Agent Canvas delivers measurable value.

Financial Services: Market Intelligence Copilot

Meridian Capital --- A mid-size hedge fund uses AI Agent Canvas to give junior analysts instant access to market data, SEC filings, and technical indicators while enforcing compliance guardrails that prevent unauthorized trading signals.

[Read the full use case →](#)

Healthcare: Clinical Research Assistant

BioNova Therapeutics --- A biotech startup streamlines clinical trial coordination by connecting their trial database through MCP, managing drug compound entities, and switching between regulatory and research personas on the fly.

[Read the full use case →](#)

Legal: Contract Review Agent

Sterling & Associates --- A law firm accelerates contract review with custom skills for clause extraction, practice-area-specific user profiles, and confidentiality guardrails that prevent sensitive terms from leaking across client matters.

[Read the full use case →](#)

E-Commerce: Customer Operations Copilot

Bloom & Vine --- A direct-to-consumer home goods brand automates order monitoring, returns processing, and escalation notifications by connecting Shopify and Zendesk through MCP and orchestrating multi-step workflows.

[Read the full use case →](#)

IT Operations: Incident Response Agent

CloudScale Systems --- A SaaS platform empowers site reliability engineers with an agent that correlates alerts from Datadog and PagerDuty, runs scheduled health checks, and enforces guardrails that prevent destructive remediation actions.

[Read the full use case →](#)

Market Intelligence Copilot

Company Profile

Meridian Capital is a mid-size hedge fund managing \$2.4 billion in assets across equity, fixed-income, and alternatives strategies. Based in Boston, the firm employs 45 investment professionals and prides itself on data-driven decision-making. Their edge depends on how fast analysts can synthesize market signals and translate them into actionable research.

The Persona

Priya Sharma, Junior Portfolio Analyst. Two years out of her MBA, Priya covers the industrials sector. Her day starts at 6:30 AM reviewing overnight market moves and ends with a stack of research notes that feed the senior portfolio managers' weekly allocation meeting. She juggles Bloomberg terminals, SEC filings, internal models, and Slack threads --- often copying data between systems manually.

The Problem

Priya spends 60% of her day on data gathering rather than analysis. Pulling a single stock's fundamentals, technicals, and recent SEC filings requires switching between four applications. When the PM asks "What changed overnight in our watchlist?", Priya needs 45 minutes to compile an answer. During earnings season, the backlog becomes unmanageable.

The compliance team adds another layer: analysts must not share raw trading signals outside approved channels, and every piece of external data used in a recommendation must be traceable to its source.

How AI Agent Canvas Solves It

Meridian deploys AI Agent Canvas as an internal copilot accessible through a CopilotKit-powered web interface embedded in their research portal.

MCP Data Connections

The team configures **MCP.MarketData** with three core tools:

- **stock_quote** --- Retrieves real-time and historical price data for any ticker. Priya asks "What's the current price and 30-day performance of CAT?" and gets a structured response with price, change, volume, and moving averages.
- **edgar_company_facts** --- Queries SEC EDGAR for company filings, financial statements, and key metrics. "Pull CAT's latest 10-K revenue breakdown by segment" returns structured data extracted from the filing.

- **stock_technicals** --- Computes technical indicators (RSI, MACD, Bollinger Bands) on demand. "Show me the RSI and MACD for CAT over the last 90 days" returns the computed values with interpretation notes.

Scheduled Tasks

Meridian configures two scheduled scans:

- **Morning Watchlist Briefing (6:00 AM ET)** --- The agent iterates through the firm's 85-stock watchlist, pulls overnight price changes, flags any stock that moved more than 2%, and checks for new SEC filings posted after market close. The summary lands in Priya's inbox before she arrives.
- **Earnings Calendar Alert (Weekly, Monday 7:00 AM)** --- Scans the upcoming week's earnings calendar against the watchlist and generates a preparation checklist for each reporting company.

Guardrails

Compliance requirements are enforced through two custom guardrails:

- **No Trading Signals** --- The agent is prohibited from generating buy/sell/hold recommendations or price targets. It can present data and flag anomalies, but the investment thesis must come from the analyst. If Priya asks "Should I buy CAT?", the agent responds with relevant data points and declines to make a recommendation.
- **Source Attribution** --- Every data point in the agent's response includes a source tag (e.g., "SEC EDGAR 10-K filed 2026-02-14" or "Market data as of 2026-06-21 09:31 ET"). This creates an audit trail that satisfies the compliance team's traceability requirements.

Personas

The agent supports two personas:

- **Research Mode** (default) --- Verbose responses with full data tables, charts descriptions, and contextual notes. Designed for deep analysis sessions.
- **Briefing Mode** --- Concise bullet-point summaries optimized for the morning standup. Priya switches to this mode when preparing materials for the PM meeting.

A Day in the Life

6:15 AM --- Priya opens her laptop to find the morning watchlist briefing already in her inbox. Three stocks moved more than 2% overnight: Caterpillar (CAT) dropped 3.1% on a supplier disruption report, Deere (DE) rose 2.4% on strong Asia-Pacific orders, and United Rentals (URI) filed an 8-K disclosing a new credit facility.

6:30 AM --- She opens the copilot and asks: "Deep dive on CAT's supplier disruption. Pull the latest 10-K supply chain risk disclosures and compare current RSI to the last three drawdowns." The agent retrieves the relevant 10-K section, computes the RSI comparison, and presents a structured analysis --- without suggesting whether the dip is a buying opportunity.

8:00 AM --- During the morning standup, the PM asks about URI's new credit facility. Priya switches the agent to Briefing Mode and asks: "Summarize URI's 8-K filed yesterday. What are the key terms?" She gets a five-bullet summary she can read aloud.

10:30 AM --- Priya is building a sector comparison. She asks: "Compare P/E, EV/EBITDA, and revenue growth for CAT, DE, URI, and PCAR using the most recent quarterly data." The agent pulls EDGAR facts for all four companies and presents a comparison table with source citations.

2:00 PM --- She drafts her weekly sector note and asks the agent to verify three specific data points against SEC filings. The agent confirms two and flags that one revenue figure she cited is from the prior quarter, not the most recent.

Key Configuration

Component	Configuration
Agents	MarketIntelligenceAgent with CanHandle matching financial queries
MCP Connections	MCP.MarketData (stock_quote, edgar_company_facts, stock_technicals)
Scheduled Tasks	Morning Watchlist Briefing (daily 6:00 AM), Earnings Calendar Alert (weekly Monday 7:00 AM)
Personas	Research Mode (verbose analysis), Briefing Mode (concise bullets)
Guardrails	NoTradingSignals, SourceAttribution
User Profiles	Per-analyst watchlist configuration, sector assignment
Entity Management	Stock tickers, SEC filing references, internal model identifiers

Clinical Research Assistant

Company Profile

BioNova Therapeutics is a 90-person biotech startup in Cambridge, Massachusetts, developing next-generation immunotherapy treatments for autoimmune diseases. The company has three compounds in clinical trials --- two in Phase II and one entering Phase III. With a lean team and aggressive timelines, every hour of manual data work delays their path to regulatory submission.

The Persona

Dr. Marcus Chen, Clinical Research Coordinator. Marcus manages the day-to-day operations of BioNova's Phase II trial for BNV-204, a monoclonal antibody targeting lupus. He coordinates across 12 clinical sites, tracks adverse events, manages regulatory submissions, and ensures protocol compliance. His background is in clinical pharmacology, but he spends most of his time wrangling spreadsheets and chasing site coordinators for overdue data.

The Problem

BioNova's clinical trial data lives in a purpose-built trial management system, but extracting insights requires SQL queries that Marcus cannot write himself. He submits data requests to the biostatistics team, waits 2-3 days for results, and often discovers the query didn't capture exactly what he needed. Meanwhile, regulatory deadlines are fixed.

Compound information is scattered across internal wikis, lab notebooks, and regulatory submission documents. When the FDA requests a specific detail about BNV-204's pharmacokinetic profile, Marcus spends hours locating the authoritative source.

The team also needs to operate in two distinct modes: exploratory research conversations (where speculation is acceptable) and regulatory-grade reporting (where every statement must be traceable and precise).

How AI Agent Canvas Solves It

BioNova deploys AI Agent Canvas as an internal research assistant accessible to the clinical operations team.

MCP Data Connections

A custom **MCP.ClinicalTrials** connector exposes the trial management database through three tools:

- **query_enrollment** --- Returns enrollment status by site, cohort, and time period. Marcus asks "How many patients completed the Week 12 visit at the Boston and Philadelphia sites?" and gets a precise count with enrollment dates.

- **query_adverse_events** --- Retrieves adverse event reports filtered by severity, system organ class, and relatedness assessment. "List all Grade 3+ adverse events in Cohort B reported in the last 30 days" returns structured records with patient identifiers redacted to study IDs.
- **query_endpoints** --- Pulls primary and secondary endpoint data for interim analysis. "What is the mean change in SLEDAI score from baseline across all active patients?" returns the computed statistic with confidence intervals.

Entity Management

BioNova uses entity management to maintain a canonical registry of their drug compounds:

- **BNV-204** --- Monoclonal antibody, anti-BAFF, Phase II lupus trial (NCT-XXXXX)
- **BNV-118** --- Small molecule, JAK1 selective inhibitor, Phase II rheumatoid arthritis
- **BNV-330** --- Bispecific antibody, Phase III psoriatic arthritis

Each entity includes structured metadata: mechanism of action, molecular target, current trial phase, IND number, and links to key regulatory documents. When Marcus asks about "the lupus compound," the agent resolves it to BNV-204 and pulls the correct context.

Personas

The agent operates in two personas, switchable by Marcus or triggered by context:

- **Research Mode** --- Allows exploratory analysis. The agent can suggest hypotheses, note trends in the data, and speculate about potential correlations. Responses include caveats marking speculative content. Used during internal team discussions and brainstorming sessions.
- **Regulatory Mode** --- Strict, citation-heavy responses. Every data point must reference its source (database query, document section, or filing). The agent declines to speculate and flags when a question cannot be answered from available data alone. Used when preparing FDA submissions, responding to information requests, or drafting study reports.

Guardrails

- **Patient Privacy** --- The agent never surfaces individual patient identifiers. All patient references use study IDs. If Marcus asks a question that would require revealing protected health information, the agent explains the restriction and suggests an aggregate alternative.
- **No Diagnostic Claims** --- The agent cannot state that a compound is effective, safe, or superior. It presents data and statistical results, but therapeutic conclusions must come from the clinical team and the Data Safety Monitoring Board.

A Day in the Life

7:30 AM --- Marcus reviews the weekly enrollment dashboard the agent compiled overnight. Two sites are behind their enrollment targets. The agent has flagged specific screen failure reasons at

each site, pulling data from the trial database.

9:00 AM --- The medical monitor asks Marcus for a summary of all serious adverse events (SAEs) reported across sites in the last quarter. Marcus opens the copilot in Regulatory Mode and asks: "List all SAEs for BNV-204 in Q1 2026, grouped by system organ class, with relatedness assessment and outcome." The agent queries the database and presents a structured table with source references to each adverse event report.

11:00 AM --- During a team meeting, a scientist asks whether there's a correlation between baseline disease severity and response at Week 12. Marcus switches to Research Mode and asks: "Is there a trend between baseline SLEDAI score and Week 12 SLEDAI change in Cohort A?" The agent runs the query, presents a summary, and notes: "This is an exploratory observation and has not been adjusted for confounders. Formal analysis should be performed by biostatistics."

2:00 PM --- The FDA sends an information request about BNV-204's pharmacokinetic profile in hepatic impairment. Marcus asks the agent: "What PK data do we have for BNV-204 in patients with mild hepatic impairment?" The agent searches the entity registry for BNV-204, locates the relevant Phase I PK study report, and extracts the relevant section --- all in Regulatory Mode with full document citations.

4:00 PM --- Marcus needs to update the site monitoring visit schedule. He asks: "Which sites have overdue monitoring visits based on the protocol-specified frequency?" The agent cross-references the visit log against the protocol schedule and identifies three sites that need attention.

Key Configuration

Component	Configuration
Agents	ClinicalResearchAgent with CanHandle matching trial, compound, and regulatory queries
MCP Connections	MCP.ClinicalTrials (query_enrollment, query_adverse_events, query_endpoints)
Personas	Research Mode (exploratory, allows speculation), Regulatory Mode (strict, citation-required)
Entity Management	Drug compound registry (BNV-204, BNV-118, BNV-330) with structured metadata
Guardrails	PatientPrivacy (no individual identifiers), NoDiagnosticClaims (no efficacy/safety conclusions)
User Profiles	Role-based access: clinical ops (full query access), research scientists (endpoint data only), regulatory affairs (all data + submission tools)
Scheduled Tasks	Weekly enrollment dashboard (Monday 6:00 AM), overdue visit alerts (daily 8:00 AM)

Contract Review Agent

Company Profile

Sterling & Associates is a 120-attorney law firm headquartered in Chicago with offices in New York and San Francisco. The firm specializes in corporate transactions, intellectual property, and commercial litigation. Their clients range from mid-market companies to Fortune 500 enterprises, and the volume of contracts flowing through the firm at any given time exceeds 400 active matters.

The Persona

Elena Vasquez, Associate Attorney. Five years into her practice, Elena works in the corporate transactions group. She reviews 15-20 contracts per week --- supplier agreements, licensing deals, NDAs, and merger-related ancillary documents. She is sharp, efficient, and frustrated. The firm's document management system holds thousands of precedent agreements, but finding the right clause from a prior deal requires searching through dozens of files manually.

The Problem

Contract review at Sterling follows a predictable but time-consuming pattern. Elena receives a draft agreement from opposing counsel, reads it cover to cover, identifies problematic clauses, cross-references the firm's standard positions, checks for client-specific requirements, and redlines the document. A routine supplier agreement takes 3-4 hours. A complex licensing deal can take two full days.

Three specific bottlenecks slow her down. First, locating precedent language: "How did we handle the limitation of liability in the Acme Industries deal last year?" requires emailing colleagues or manually searching the DMS. Second, client-specific requirements are tracked in internal memos that are not linked to the DMS, so Elena must remember or re-discover each client's preferences. Third, confidentiality walls between practice groups mean she must be careful not to access or reference documents from matters she is not staffed on.

How AI Agent Canvas Solves It

Sterling deploys AI Agent Canvas as a contract review assistant integrated into their attorney workspace.

MCP Data Connections

A custom **MCP.DocumentManagement** connector interfaces with the firm's iManage DMS:

- **search_precedents** --- Searches the DMS for contracts matching specified criteria: clause type, client, industry, deal type, and date range. Elena asks "Find indemnification clauses from technology licensing agreements closed in the last 18 months" and receives a ranked list of matching clauses with document references.

- **retrieve_clause** --- Extracts a specific clause from a document by section heading or clause type. "Pull the limitation of liability from the Acme Industries MSA (matter 2025-1847)" returns the exact language with its document citation.
- **check_client_standards** --- Queries the client standards database to retrieve the firm's approved positions for a specific client. "What is our standard position on consequential damages for TechCorp?" returns the approved language and any client-specific notes.

Skill Authoring

The firm creates custom skills that encode their contract review methodology:

- **Clause Extraction** --- Given a contract document, identifies and categorizes every material clause: indemnification, limitation of liability, termination, assignment, change of control, IP ownership, confidentiality, and governing law. Each extracted clause is tagged with a risk assessment (standard, requires review, non-standard).
- **Redline Comparison** --- Compares a received draft against the firm's standard template for that contract type and highlights deviations. The output is a structured list of differences with the firm's preferred alternative language.
- **Position Reconciliation** --- Cross-references extracted clauses against client-specific standards and flags any clause where the draft deviates from the client's approved position.

Guardrails

- **Matter-Based Access Control** --- The agent enforces confidentiality walls by restricting DMS searches to matters where the requesting attorney is listed as a team member. If Elena searches for precedents, she only sees results from her own matters and from the firm's general precedent library. Documents from walled-off matters are excluded from results.
- **No Legal Advice Framing** --- The agent presents its analysis as draft work product for attorney review. Every output includes a disclaimer that the analysis has not been reviewed by a supervising attorney and does not constitute the firm's legal opinion.
- **Privilege Protection** --- The agent will not include attorney-client privileged content in any output that could be shared externally. If Elena asks the agent to draft a clause summary for opposing counsel, it strips internal notes and work product references.

User Profiles

User profiles map each attorney to their practice area, seniority level, and matter assignments:

- **Practice Area** determines which standard templates and clause libraries are loaded by default. Elena's corporate transactions profile loads commercial contract templates. An IP attorney's profile loads licensing and patent agreement templates.
- **Seniority Level** controls the depth of analysis. Associates receive detailed explanations and citations to precedent. Partners receive concise summaries with only flagged deviations.
- **Matter Assignments** define the DMS access perimeter for confidentiality wall enforcement.

A Day in the Life

8:00 AM --- Elena receives a draft Master Services Agreement from opposing counsel for a new TechCorp engagement. She uploads the document to the copilot and asks: "Extract all material clauses and assess against our standard MSA template."

8:15 AM --- The agent returns a structured analysis. Fourteen clauses extracted. Three flagged as non-standard: the indemnification cap is set at contract value (the firm's standard is 2x), the termination for convenience requires 90-day notice (standard is 30), and the IP ownership clause contains a broad license-back provision not present in the template.

8:30 AM --- Elena asks: "Check these three flagged clauses against TechCorp's client standards." The agent queries the client standards database and reports: TechCorp has previously accepted a 1.5x indemnification cap, requires no more than 60-day termination notice, and has no documented position on license-back provisions. Elena now knows exactly where she has room to negotiate and where she needs partner guidance.

10:00 AM --- For the license-back provision, Elena needs precedent. She asks: "Find license-back clauses from technology MSAs where we successfully negotiated narrower scope, last two years." The agent returns four examples from her accessible matters, each with the original draft language and the final negotiated version.

11:30 AM --- Elena drafts her redline and asks the agent to verify consistency: "Check my redlined indemnification clause against the TechCorp standard and flag any conflicts." The agent confirms alignment and notes that her proposed cap of 1.5x matches the client's documented threshold.

3:00 PM --- A partner asks Elena to prepare a summary of open issues for the client call tomorrow. Elena asks the agent: "Summarize the three non-standard clauses, our proposed positions, and the precedent supporting each." The agent produces a one-page brief with citations to the precedent agreements --- all from matters Elena is authorized to access.

Key Configuration

Component	Configuration
Agents	ContractReviewAgent with CanHandle matching contract, clause, and agreement queries
MCP Connections	MCP.DocumentManagement (search_precedents, retrieve_clause, check_client_standards)
Skills	ClauseExtraction, RedlineComparison, PositionReconciliation
Guardrails	MatterBasedAccessControl, NoLegalAdviceFraming, PrivilegeProtection
User Profiles	Per-attorney: practice area, seniority level, matter assignments
Personas	Associate Mode (detailed analysis with citations), Partner Mode (concise flagged deviations)
Entity Management	Client registry, matter database, standard clause library

Customer Operations Copilot

Company Profile

Bloom & Vine is a direct-to-consumer home goods brand based in Portland, Oregon. Founded four years ago, the company sells handcrafted ceramics, textiles, and sustainable home decor through their Shopify storefront. Annual revenue hit \$18 million last year, driven by a loyal customer base and strong social media presence. The operations team is small --- eight people handling everything from order fulfillment to customer support.

The Persona

Aisha Thompson, Customer Operations Manager. Aisha oversees the end-to-end customer experience: order processing, shipping logistics, returns, and support ticket resolution. She manages a team of three support agents and personally handles escalated issues. Her tools include Shopify for orders, Zendesk for support tickets, ShipStation for fulfillment tracking, and a spreadsheet she maintains for tracking recurring issues. She is the connective tissue between the warehouse, the support team, and the founders.

The Problem

Bloom & Vine's growth has outpaced their operational tooling. Aisha's team handles 200+ support tickets per week across email, chat, and social channels. The most common issues --- order status inquiries, return requests, and shipping delays --- are straightforward but time-consuming because each requires switching between Shopify, Zendesk, and ShipStation to gather context.

Returns processing is particularly painful. The company's return policy varies by product category (ceramics have a different breakage policy than textiles), and the support agents frequently need to check the policy, verify the order, assess the reason, and determine whether to issue a refund, replacement, or store credit. Each return takes 12-15 minutes to process manually.

Peak seasons are worse. During the holiday rush, the ticket backlog grows to 500+ and response times stretch beyond 48 hours. Aisha spends her evenings triaging tickets instead of improving processes.

How AI Agent Canvas Solves It

Bloom & Vine deploys AI Agent Canvas as an internal operations copilot that Aisha and her support agents use alongside their existing tools.

MCP Data Connections

Two custom MCP connectors bridge the key operational systems:

MCP.Shopify exposes order and product data:

- **lookup_order** --- Retrieves complete order details by order number, email, or customer name. Includes line items, payment status, fulfillment status, shipping tracking, and order history. "Pull up order #BV-28491" returns everything Aisha needs in one response.
- **lookup_customer** --- Returns customer profile with order history, lifetime value, return history, and any internal notes. "Show me the customer profile for sarah.chen@email.com" reveals that Sarah is a repeat buyer with \$2,400 in lifetime purchases and zero prior returns.
- **process_return** --- Initiates a return in Shopify: creates the return record, generates a shipping label, and triggers the appropriate refund type (original payment, store credit, or replacement). Requires explicit confirmation before executing.

MCP.Zendesk connects the support ticket system:

- **get_ticket** --- Retrieves a Zendesk ticket with full conversation history, tags, and internal notes. "What's the status of ticket #41872?" returns the complete thread.
- **update_ticket** --- Adds internal notes, changes priority, assigns to a team member, or updates tags. Used by the agent to enrich tickets with order context automatically.
- **search_tickets** --- Finds tickets matching criteria: status, assignee, creation date, tags, or keyword. "Show me all open tickets tagged 'shipping-delay' from the last 7 days" returns a filtered list.

Scheduled Tasks

Automated monitoring runs continuously in the background:

- **Order Anomaly Scan (Every 2 hours)** --- Checks all orders placed in the last 48 hours for anomalies: payment failures, fulfillment delays beyond SLA, duplicate orders, and address validation warnings. Flagged orders appear in a daily digest or trigger immediate notifications for high-severity issues.
- **Shipping Delay Monitor (Every 4 hours)** --- Cross-references ShipStation tracking data against expected delivery dates. Orders that are delayed beyond the carrier's estimate by more than 2 days are flagged, and the agent pre-drafts a proactive customer communication.
- **Weekly Ticket Trends (Monday 7:00 AM)** --- Analyzes the prior week's ticket volume, resolution time, and category distribution. Identifies emerging patterns --- for example, a spike in "item arrived damaged" tickets from a specific carrier.

Workflows

Multi-step workflows handle common operations end to end:

Returns Processing Workflow:

1. Support agent receives a return request and asks the copilot: "Process a return for order #BV-28491, reason: item damaged in shipping."
2. The agent looks up the order, identifies the product category (ceramics), retrieves the applicable return policy (full refund or replacement for damaged ceramics within 30 days), and verifies the order date is within the return window.

3. The agent presents the recommended action: "Order is eligible for full refund or free replacement under the ceramics breakage policy. Customer has zero prior returns. Recommended: offer both options to the customer."
4. Upon confirmation, the agent creates the return in Shopify, generates the prepaid return label, and drafts a response for the support agent to send through Zendesk.

Notifications and Escalations

The agent monitors for conditions that require immediate human attention:

- **VIP Customer Issues** --- Tickets from customers with lifetime value above \$1,000 are automatically escalated to Aisha with full order context attached.
- **Repeat Complaints** --- If the same customer opens a third ticket within 30 days, the agent flags the pattern and recommends a personal outreach from Aisha.
- **Inventory-Impacting Returns** --- If returns for a specific product exceed 5% of units sold in a rolling 30-day window, the agent alerts the product team with a summary of return reasons.

A Day in the Life

7:30 AM --- Aisha opens the copilot to find the overnight anomaly scan results. Two orders flagged: one payment retry failed after three attempts, and one order shipped to an undeliverable address (carrier returned the package). The agent has already pre-drafted a customer email for the address issue.

8:15 AM --- A support agent asks the copilot: "Customer says her ceramic vase arrived broken. Order #BV-31205." The agent pulls the order (placed 8 days ago, ceramics category, first-time buyer), confirms it falls within the 30-day breakage policy, and recommends a full replacement with expedited shipping to recover the customer experience. The agent drafts the Zendesk response and waits for the support agent to approve before sending.

10:00 AM --- The shipping delay monitor flags 12 orders stuck at a regional carrier hub in Dallas for 3+ days. The agent groups them by carrier route, identifies the likely cause (weather-related delays affecting the Southwest corridor), and drafts a batch notification for affected customers offering a 15% discount on their next order.

1:00 PM --- Aisha asks: "Show me the ticket trends from last week." The agent presents the weekly analysis: ticket volume was up 22% from the prior week, driven by a 40% increase in "where is my order" inquiries. The root cause correlates with the Dallas carrier delays. Average resolution time held steady at 4.2 hours because the agent automated initial order lookups for the support team.

3:30 PM --- A VIP escalation arrives. A customer with \$3,200 in lifetime purchases received the wrong item. The agent surfaces the full context: order details, what was ordered vs. what shipped, warehouse batch information, and the customer's previous positive interactions. Aisha handles the call personally, armed with everything she needs.

Key Configuration

Component	Configuration
Agents	CustomerOpsAgent with CanHandle matching order, return, shipping, and ticket queries
MCP Connections	MCP.Shopify (lookup_order, lookup_customer, process_return), MCP.Zendesk (get_ticket, update_ticket, search_tickets)
Scheduled Tasks	Order Anomaly Scan (every 2 hours), Shipping Delay Monitor (every 4 hours), Weekly Ticket Trends (Monday 7:00 AM)
Workflows	ReturnsProcessing (policy check, return creation, label generation, response drafting)
Guardrails	RequireConfirmation (all refunds and replacements require human approval), SpendLimit (refunds above \$200 escalate to Aisha)
User Profiles	Support agents (ticket and order access), Aisha (full access + escalation management), warehouse team (fulfillment data only)
Entity Management	Product catalog with category-specific return policies, carrier routing rules

Incident Response Agent

Company Profile

CloudScale Systems is a B2B SaaS platform providing real-time inventory management to mid-market retailers. The platform processes 2.3 million API requests per hour across three AWS regions, serving 340 enterprise customers who depend on sub-second response times for their point-of-sale and e-commerce integrations. The engineering team is 55 people, with a 6-person SRE team responsible for platform reliability.

The Persona

Jordan Park, Site Reliability Engineer. Jordan has been an SRE at CloudScale for three years. They are on the on-call rotation every third week and responsible for maintaining the platform's 99.95% uptime SLA. Their toolkit includes Datadog for observability, PagerDuty for alerting, Terraform for infrastructure, and a growing collection of runbooks in Confluence that are perpetually half-updated. Jordan is technically strong but drowning in alert noise --- on a typical on-call shift, they receive 60-80 alerts, of which fewer than 10 require human action.

The Problem

CloudScale's monitoring stack generates high-quality telemetry, but correlating signals across systems during an incident is entirely manual. When a PagerDuty alert fires at 2 AM, Jordan's workflow looks like this: acknowledge the alert, open Datadog, check the relevant dashboard, correlate with recent deployments, check dependent services, look at the database metrics, review the last similar incident's postmortem, and decide whether to page the application team or handle it directly.

This triage process takes 10-20 minutes per incident --- time that matters when the SLA clock is ticking. Worse, during cascading failures, multiple alerts fire simultaneously, and Jordan must mentally model dependencies to determine root cause versus symptom.

The team also struggles with runbook discipline. Runbooks exist for the 15 most common incident types, but they drift out of date as the architecture evolves. During an incident, Jordan often discovers the runbook references a service that was renamed or a metric that was replaced.

Finally, some remediation actions are dangerous. Restarting a service is safe; scaling down a database cluster is not. The team needs guardrails that prevent well-intentioned but destructive actions during the pressure of a live incident.

How AI Agent Canvas Solves It

CloudScale deploys AI Agent Canvas as an incident response copilot accessible to the SRE team through their internal tools portal and a dedicated Slack integration.

MCP Data Connections

A custom **MCP.Observability** connector bridges the monitoring stack:

- **query_metrics** --- Retrieves time-series metrics from Datadog by service, metric name, and time range. "Show me the p99 latency for the inventory-api service over the last 2 hours" returns the metric data with trend analysis and anomaly detection.
- **get_alerts** --- Pulls active and recent alerts from PagerDuty, enriched with Datadog context. "What alerts fired in the last 30 minutes?" returns a correlated view: the PagerDuty alert, the triggering Datadog monitor, the affected service, and related metrics.
- **get_deployment_history** --- Queries the deployment pipeline for recent releases. "What was deployed to production in the last 6 hours?" returns a chronological list of deployments with service name, version, deployer, and commit hash.
- **query_logs** --- Searches centralized logs by service, severity, time range, and keyword. "Show me error logs from order-processor in the last 15 minutes" returns matching log entries with context lines.
- **get_service_dependencies** --- Returns the dependency graph for a given service. "What depends on the inventory-database service?" maps upstream and downstream dependencies, helping Jordan understand blast radius.

Scheduled Health Checks

Proactive monitoring runs on automated schedules:

- **SLA Budget Check (Every 15 minutes)** --- Calculates remaining error budget for the current month based on actual downtime and degraded performance windows. When the budget drops below 30%, the agent sends a notification to the SRE channel with a breakdown of budget-consuming incidents.
- **Deployment Correlation Scan (Every 30 minutes)** --- After each production deployment, monitors the deployed service's error rate and latency for 30 minutes. If metrics deviate beyond baseline thresholds, the agent alerts the deployer and the on-call SRE with a pre-built correlation report.
- **Dependency Health Sweep (Every hour)** --- Checks health endpoints and key metrics for all critical-path services. Surfaces degradation trends before they trigger customer-facing alerts.
- **Certificate and Secret Expiry (Daily 6:00 AM)** --- Scans for TLS certificates and API keys approaching expiration within 30 days and creates tickets in the backlog.

Guardrails

Safety controls prevent destructive actions during high-stress incidents:

- **Read-Only by Default** --- The agent can query any metric, log, or alert but cannot execute remediation actions (restart services, scale infrastructure, modify configurations) without explicit human approval. Every proposed action is presented as a recommendation, not an execution.

- **Destructive Action Blocklist** --- Certain actions are permanently blocked regardless of approval: scaling down database clusters, modifying network security groups, deleting persistent volumes, and rotating production credentials. These actions must be performed through the standard change management process.
- **Blast Radius Assessment** --- Before recommending any remediation action, the agent queries the service dependency graph and reports the blast radius. "Restarting inventory-api will affect 3 downstream services (order-processor, fulfillment-engine, analytics-pipeline) serving approximately 800 requests/minute. Proceed?"
- **Incident Escalation Rules** --- If the agent detects a Severity 1 incident (multiple services affected, SLA impact), it automatically pages the incident commander and creates a structured incident channel --- but cannot acknowledge the page on Jordan's behalf.

Entity Management

The service catalog is maintained as managed entities:

- Each service has a registered entry: name, team owner, tier (critical/standard/best-effort), SLA target, deployment pipeline, runbook link, and dependency list.
- The agent uses this catalog to contextualize every interaction. When Jordan asks about "the inventory service," the agent resolves it to `inventory-api` (Tier 1, owned by the Platform team, 99.99% SLA target) and pulls the relevant runbook.
- When runbook steps reference outdated service names or deprecated metrics, the agent flags the discrepancy and suggests an update.

A Day in the Life

2:14 AM --- Jordan's phone buzzes. PagerDuty alert: "High error rate on order-processor (>5% 5xx responses, 8-minute duration)." Jordan opens the copilot from their phone and asks: "What's happening with order-processor?"

2:15 AM --- The agent correlates in real time. It pulls the PagerDuty alert, the Datadog error rate metric (currently 7.2%, baseline 0.3%), recent logs (connection timeout errors to inventory-database), the dependency graph (order-processor depends on inventory-database), and inventory-database metrics (connection pool saturation at 98%). The agent presents: "Root cause appears to be connection pool exhaustion on inventory-database. Order-processor is failing because it cannot acquire database connections. Three other services (fulfillment-engine, analytics-pipeline, pricing-service) also depend on inventory-database but are not yet showing errors."

2:17 AM --- Jordan asks: "Was anything deployed recently?" The agent reports: "inventory-database connection pool configuration was updated 47 minutes ago by deploy pipeline. The `max_connections` parameter was changed from 200 to 100. This correlates with the onset of connection pool saturation."

2:19 AM --- Jordan asks: "Can we roll back that configuration change?" The agent responds: "I can recommend the rollback but cannot execute it directly. The rollback would restore max_connections to 200. Blast radius assessment: this change affects inventory-database only. No downstream services would be disrupted by the rollback --- in fact, it should resolve the current incident. Recommended: execute rollback through the deployment pipeline."

2:22 AM --- Jordan triggers the rollback through the standard pipeline. The agent monitors the recovery: connection pool utilization drops to 45% within 3 minutes, order-processor error rate returns to baseline within 5 minutes. The agent confirms: "Incident resolved. Error rate normalized. No SLA impact --- total elevated error window was 19 minutes against a 43-minute monthly budget remaining."

2:30 AM --- The agent drafts a preliminary postmortem with timeline, root cause, remediation steps, and a suggested action item: "Add a guardrail to the deployment pipeline that prevents connection pool reductions greater than 25% without SRE approval."

9:00 AM --- Jordan reviews the overnight health check results. The certificate expiry scan flagged two API keys expiring in 12 days. The dependency health sweep shows a gradual increase in p99 latency on the pricing-service over the last week --- not yet alerting, but trending toward the threshold. Jordan opens an investigation ticket.

Key Configuration

Component	Configuration
Agents	IncidentResponseAgent with CanHandle matching alert, incident, deployment, and service queries
MCP Connections	MCP.Observability (query_metrics, get_alerts, get_deployment_history, query_logs, get_service_dependencies)
Scheduled Tasks	SLA Budget Check (every 15 min), Deployment Correlation Scan (every 30 min), Dependency Health Sweep (hourly), Certificate/Secret Expiry (daily 6:00 AM)
Guardrails	ReadOnlyDefault (no auto-remediation), DestructiveActionBlocklist (permanent blocks on dangerous operations), BlastRadiusAssessment (dependency impact before any action), IncidentEscalationRules (auto-page for Sev1)
Entity Management	Service catalog with tier, owner, SLA target, dependencies, and runbook links
User Profiles	SRE team (full observability access + remediation recommendations), application developers (service-scoped metrics and logs), management (SLA dashboards and incident summaries)
Personas	Incident Mode (terse, action-oriented, correlates signals), Analysis Mode (detailed, trend-focused, for post-incident review)

User Guide

Installation, configuration, and usage of every feature including chat, personas, skills, workflows, scheduling, and security.

Getting Started

This guide walks you through installing, configuring, and running AI Agent Canvas for the first time.

Prerequisites

Before you begin, make sure you have the following installed:

- **.NET 9 SDK** -- [Download from Microsoft](#)
- **Node.js 18+** -- [Download from nodejs.org](#)
- **Azure AI Foundry account** (or any OpenAI-compatible endpoint) -- You need an endpoint URL, an API key, and a model deployment name.

Verify your installations:

```
dotnet --version # Should print 9.x
node --version # Should print v18.x or higher
npm --version # Should print 9.x or higher
```

Clone the Repository

```
git clone <your-repo-url> AgentOpsHub
cd AgentOpsHub
```

Build the Backend

From the repository root, restore packages and build the solution:

```
dotnet build AiAgentCanvas.sln
```

A successful build compiles the core framework, agents, MCP connections, scheduler, skills engine, and the web host.

Configure Your AI Endpoint

Open `src/AiAgentCanvas.Web/appsettings.json` and fill in the `AIFoundry` section with your endpoint details:

```
{
  "AIFoundry": {
    "Endpoint": "https://your-resource.openai.azure.com/",
    "Key": "your-api-key-here",
    "DeploymentName": "gpt-4o",
    "UseAzureCredential": false
  }
}
```

Field	Description
-------	-------------

Endpoint	Your Azure AI Foundry (or OpenAI-compatible) endpoint URL.
Key	The API key for authentication.
DeploymentName	The model deployment to use (e.g., gpt-4o).
UseAzureCredential	Set to true to use Azure Managed Identity instead of an API key.

See [Configuration](#) for the full settings reference.

Run the Backend

Start the .NET backend server:

```
cd src/AiAgentCanvas.Web
dotnet run
```

The backend starts on `http://localhost:5000` by default. You should see log output confirming that the agent framework, scheduler, and tool providers have loaded.

Run the Frontend

In a separate terminal, install dependencies and start the Next.js development server:

```
cd frontend
npm install
npm run dev
```

The frontend starts on `http://localhost:3000`.

Open the Chat Interface

Open your browser and navigate to **`http://localhost:3000`**. You will see the CopilotKit chat interface.

Your First Conversation

Type a message in the chat input and press Enter. The agent processes your message, and you will see the response stream in token by token.

Try these to explore the main features:

1. **Ask a question** -- Type "What can you help me with?" to see the agent describe its capabilities.
2. **Use a built-in tool** -- Type "Get me a stock quote for AAPL" to see the agent call the `stock_quote` tool and return live data.
3. **Create a persona** -- Type "Create a persona called Analyst with instructions to focus on data-driven insights." See [Personas](#) for details.

4. **Schedule a task** -- Type "Schedule a recurring task every hour to check the S&P 500 price." See [Scheduling](#) for details.
5. **Create a workflow** -- Type "Create a workflow called Morning Briefing that gets stock quotes for AAPL, MSFT, and GOOGL." See [Workflows](#) for details.

Project Structure at a Glance

Folder	Purpose
src/AiAgentCanvas/	Core framework (agent orchestration, AG-UI protocol, tool registry)
src/MyAgents/	Custom agent logic
src/MCP/	Data connections (MCP clients)
src/AiAgentCanvas.Web/	Composition root -- wires everything together
frontend/	Next.js + CopilotKit chat UI
agent-data/	Runtime data (personas, workflows, entities, guardrails)

Next Steps

- [Configuration](#) -- Detailed settings reference
- [Chat Interface](#) -- How the chat UI works
- [Skills & Tools](#) -- Built-in and custom tools
- [Security](#) -- Governance policies and rate limiting

Configuration

All backend configuration lives in `src/AiAgentCanvas.Web/appsettings.json`. This page covers every section and how to customize it for your environment.

Configuration File Structure

The main configuration file contains these sections:

```
{
  "Logging": { ... },
  "AllowedHosts": "*",
  "AIFoundry": { ... },
  "Security": { ... },
  "AlphaVantage": { ... }
}
```

AIFoundry Section

This section configures the connection to your LLM provider. AI Agent Canvas uses Azure AI Foundry by default but works with any OpenAI-compatible endpoint.

```
{
  "AIFoundry": {
    "Endpoint": "https://your-resource.openai.azure.com/",
    "Key": "your-api-key-here",
    "DeploymentName": "gpt-4o",
    "UseAzureCredential": false
  }
}
```

Setting	Required	Description
Endpoint	Yes	The base URL of your AI endpoint.
Key	Yes*	API key for authentication. Not required if UseAzureCredential is true.
DeploymentName	Yes	The model deployment name (e.g., gpt-4o, gpt-4o-mini).
UseAzureCredential	No	Set to true to authenticate with Azure Managed Identity instead of an API key. Defaults to false.

Azure AI Foundry Setup

1. Create an Azure AI Foundry resource in the Azure Portal.
2. Deploy a model (e.g., gpt-4o) and note the deployment name.
3. Copy the endpoint URL and API key from the resource's Keys and Endpoint page.
4. Paste them into the AIFoundry section.

Using Azure Managed Identity

For production deployments, use Managed Identity instead of API keys:

```
{
  "AIFoundry": {
    "Endpoint": "https://your-resource.openai.azure.com/",
    "Key": "",
    "DeploymentName": "gpt-4o",
    "UseAzureCredential": true
  }
}
```

Ensure the hosting identity has the **Cognitive Services OpenAI User** role on the AI Foundry resource.

Security Section

Controls governance policies and rate limiting.

```
{
  "Security": {
    "PolicyPath": "governance-policy.yaml",
    "RateLimitPerMinute": 30
  }
}
```

Setting	Default	Description
PolicyPath	governance-policy.yaml	Path to the YAML file defining governance rules. See Security.
RateLimitPerMinute	30	Maximum API requests per minute per client. Returns HTTP 429 when exceeded.

AlphaVantage Section

Configures the Alpha Vantage API for market data tools (stock quotes, technicals).

```
{
  "AlphaVantage": {
    "ApiKey": "demo"
  }
}
```

Setting	Description
ApiKey	Your Alpha Vantage API key. Use "demo" for limited testing. Get a free key at alphavantage.co .

Logging Section

Standard .NET logging configuration.

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  }
}
```

Raise the default level to Debug for troubleshooting. Set `Microsoft.AspNetCore` to Information to see HTTP request logs.

Environment-Specific Configuration

Create environment-specific overrides using the standard ASP.NET Core pattern:

- `appsettings.Development.json` -- Applied when `ASPNETCORE_ENVIRONMENT=Development` (the default during `dotnet run`).
- `appsettings.Production.json` -- Applied in production deployments.

Example `appsettings.Development.json`:

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Debug"
    }
  },
  "AIFoundry": {
    "Key": "dev-only-key"
  }
}
```

Environment-specific files override values from the base `appsettings.json`. You do not need to repeat settings that stay the same.

Environment Variables

Any setting can be overridden via environment variables using the `__` (double underscore) separator:

```
export AIFoundry__Key="your-key-here"
export Security__RateLimitPerMinute=60
```

This is useful for CI/CD pipelines and container deployments where you do not want secrets in config files.

Frontend Configuration

The frontend reads its backend URL from `frontend/.env.local`:

```
NEXT_PUBLIC_BACKEND_URL=http://localhost:5000
```

Change this when the backend runs on a different host or port.

Runtime Data Directories

The backend creates and uses these directories at runtime (relative to the working directory):

Directory	Purpose
agent-data/personas/	Persona definitions (markdown files)
agent-data/workflows/	Workflow definitions (markdown files)
agent-data/guardrails/	Guardrail rules (markdown files)
agent-data/entities/	Entity memory (markdown files)
agent-data/profiles/	User profiles (markdown files)
agent-data/context/	Persistent context entries (markdown files)
agent-data/skills/	Locally authored skill definitions
skills.db	SQLite database for skill records
hangfire.db	SQLite database for scheduled task state

These directories are created automatically on first use. No manual setup is required.

Chat Interface

AI Agent Canvas uses [CopilotKit](#) to provide a real-time chat interface where you interact with the agent. This page explains how the chat UI works and what to expect.

Overview

The chat interface is a single-page application built with Next.js and the CopilotKit React components. It connects to the backend via the AG-UI protocol (Server-Sent Events) at the `/api/copilotkit` endpoint.

When you open **`http://localhost:3000`** in your browser, you see a full-screen chat panel ready for input.

Sending Messages

Type your message in the input field at the bottom of the chat panel and press **Enter** to send. The agent receives your message, processes it, and streams back a response.

You can send:

- **Questions** -- "What is the current price of AAPL?"
- **Commands** -- "Create a persona called Analyst."
- **Multi-step requests** -- "Get stock quotes for AAPL, MSFT, and GOOGL, then summarize the trends."

Streaming Responses

Responses stream in token by token as the LLM generates them. You see text appearing in real time rather than waiting for the full response to complete. This gives immediate feedback that the agent is working.

Long responses may take several seconds to complete. The streaming indicator shows that generation is still in progress.

Tool Calls

When the agent needs to perform an action -- such as fetching a stock quote, creating a persona, or scheduling a task -- it makes a **tool call**. Tool calls appear in the chat as the agent transparently shows what action it is taking.

A typical tool call flow:

1. You send a message: "Get a stock quote for MSFT."
2. The agent decides to call the `stock_quote` tool.

3. The tool executes and returns data.
4. The agent incorporates the result into its response.

The agent has access to 40+ built-in tools spanning market data, personas, skills, workflows, scheduling, entities, guardrails, user profiles, and MCP connections. See [Skills & Tools](#) for details.

Conversation Context

The agent maintains context throughout a conversation. It remembers what you discussed earlier in the session and can reference previous messages, tool results, and decisions.

Several systems contribute to the agent's context:

Context Source	What It Provides
Conversation history	All messages and tool results from the current session
Active persona	Custom instructions from the currently selected persona
Active user profile	Your name, role, timezone, and preferences
Persistent context	Saved context entries that carry across sessions
Entity memory	Known entities (people, companies, projects)
Guardrail rules	Active safety rules that shape behavior

These context sources are injected into the agent's system prompt automatically. You do not need to repeat information that is already stored.

Notifications

The backend can push real-time notifications to the frontend via Server-Sent Events (SSE) at `/api/notifications`. Scheduled tasks and background processes use this channel to report results without interrupting your current conversation.

Tips for Effective Conversations

- **Be specific** -- "Get the stock quote for AAPL" works better than "tell me about Apple."
- **Chain requests** -- The agent can handle multi-step instructions in a single message.
- **Reference previous context** -- "Now do the same for MSFT" works when the agent has context from an earlier request.
- **Use natural language for management tasks** -- "List my personas," "Show scheduled tasks," and "What workflows do I have?" all work as expected.
- **Save important information** -- Ask the agent to save entities or context entries for facts you want it to remember across sessions.

Troubleshooting

Problem	Solution
Chat loads but messages fail	Check that the backend is running on port 5000. Verify NEXT_PUBLIC_BACKEND_URL in frontend/.env.local.
No response appears	Check the backend terminal for errors. Verify your AIFoundry settings in appsettings.json.
Responses are slow	Large models take longer. Consider using a faster deployment (e.g., gpt-4o-mini). Check your rate limit settings.
Tool calls fail	Check the backend logs for the specific tool error. Ensure required API keys (e.g., AlphaVantage) are configured.
Connection refused	Make sure both the backend (dotnet run) and frontend (npm run dev) are running.

Architecture Note

The frontend is intentionally thin. It renders the CopilotKit chat component and points it at the backend's AG-UI endpoint. All agent logic, tool execution, and context management happen server-side. This means you can swap or customize the frontend without changing any agent behavior.

Personas

Personas let you switch the agent's personality, tone, and focus area without changing any code. Each persona is a set of custom instructions that shape how the agent responds.

What Is a Persona?

A persona is a named configuration with:

- **Name** -- A short identifier (e.g., "Analyst", "Advisor").
- **Description** -- What the persona is for.
- **Instructions** -- Detailed instructions injected into the agent's system prompt.

When a persona is active, its instructions are prepended to every agent response, guiding the agent's behavior, tone, and areas of focus. When no persona is active, the agent uses its default system prompt.

Creating a Persona

Ask the agent to create one in natural language:

```
Create a persona called "Financial Analyst" with instructions to focus on
quantitative analysis, cite data sources, use precise financial terminology,
and present findings in a structured format with bullet points.
```

The agent calls the `create_persona` tool and saves the persona as a markdown file in `agent-data/personas/`.

Listing Personas

To see all available personas:

```
List my personas
```

The agent calls `list_personas` and returns the name and description of each persona.

Switching Personas

To activate a different persona:

```
Switch to the Financial Analyst persona
```

The agent calls `switch_persona`, and all subsequent responses in the session use that persona's instructions. The active persona is tracked in an `.active` file, so it persists across backend restarts.

Reading a Persona

To view the full details of a persona:

```
Show me the Financial Analyst persona
```

The agent calls `read_persona` and displays the persona's name, description, and instructions.

Updating a Persona

To change a persona's instructions:

```
Update the Financial Analyst persona to also include risk assessment in every analysis
```

The agent calls `update_persona` and saves the changes.

Deleting a Persona

To remove a persona:

```
Delete the Financial Analyst persona
```

The agent calls `delete_persona` and removes the markdown file.

How Personas Work Internally

Personas are managed by three components:

1. **PersonaStore** -- Reads and writes persona markdown files in `agent-data/personas/`. Each persona is a separate `.md` file.
2. **PersonaToolProvider** -- Exposes the six persona tools (`create_persona`, `update_persona`, `list_personas`, `switch_persona`, `read_persona`, `delete_persona`) to the agent.
3. **PersonaContextProvider** -- An AI context provider that reads the active persona's instructions and injects them into the agent's system prompt before every interaction.

Example Personas

Here are some useful persona configurations to get started:

Data Analyst

Create a persona called "Data Analyst" with instructions:

- Focus on quantitative analysis and data interpretation
- Always ask for data sources and sample sizes
- Present findings with tables and bullet points
- Flag statistical anomalies and outliers

Executive Advisor

Create a persona called "Executive Advisor" with instructions:

- Provide concise, strategic recommendations
- Lead with the bottom line, then support with details

- Frame insights in terms of business impact and ROI
- Use professional but accessible language

Research Assistant

Create a persona called "Research Assistant" with instructions:

- Be thorough and cite sources whenever possible
- Present multiple perspectives on complex topics
- Organize findings with clear headings and sections
- Flag areas where information is uncertain or incomplete

Tips

- **Only one persona is active at a time.** Switching personas replaces the previous one.
- **Persona instructions stack with other context.** Guardrails, user profiles, entities, and persistent context are still active alongside the persona.
- **Clear the persona** by asking "Switch to the default persona" or "Deactivate the current persona."
- **Keep instructions focused.** Shorter, clearer instructions produce more consistent behavior than long, complex ones.

Skills & Tools

AI Agent Canvas comes with built-in tools and lets you create custom skills and connect external data sources via MCP. This page covers all three.

Built-In Tools

The following tools are available out of the box when the MarketData MCP module is enabled:

Tool	Description
stock_quote	Fetches the current stock price and key metrics for a given ticker symbol.
edgar_company_facts	Retrieves SEC EDGAR filing data for a company.
stock_technicals	Returns technical analysis indicators for a stock.

Use them by asking the agent in natural language:

```
Get a stock quote for AAPL
Show me the EDGAR company facts for Microsoft
What are the technical indicators for TSLA?
```

The agent selects the appropriate tool, calls it, and incorporates the results into its response.

What Are Skills?

Skills are user-defined tool definitions that extend the agent's capabilities. Unlike built-in tools, which are compiled into the application, skills are created at runtime through conversation.

Each skill has:

- **Name** -- A short identifier.
- **Description** -- What the skill does.
- **Prompt template** -- Instructions the agent follows when the skill is invoked.

When you run a skill, the agent executes the prompt template through the LLM, producing a result based on the skill's instructions.

Creating Skills

Ask the agent to create a skill:

```
Create a skill called "Market Summary" that analyzes the current prices of
AAPL, MSFT, GOOGL, and AMZN and produces a one-paragraph market summary.
```

The agent calls `create_skill` and stores the skill in a SQLite database (`skills.db`).

Listing and Running Skills

List your skills:

```
List my skills
```

Run a skill by name:

```
Run the Market Summary skill
```

The agent calls `run_skill`, which executes the skill's prompt template through the LLM and returns the result.

Removing Skills

```
Remove the Market Summary skill
```

Skill Authoring (Markdown-Based)

For more structured skill definitions, use the skill authoring system. Authored skills are saved as markdown files with YAML frontmatter in `agent-data/skills/`:

```
Author a skill called "Competitive Analysis" with description "Analyze competitors in a given market segment" and tags "analysis, research"
```

The agent calls `author_skill` and creates a `.md` file. You can also edit and read authored skills:

```
Edit the Competitive Analysis skill to include pricing comparison
Show me the Competitive Analysis skill
Delete the Competitive Analysis skill
```

Skill Registry (Catalog)

The skill registry provides a searchable catalog of pre-built skill templates. Browse and install skills from the catalog:

```
Search available skills for "analysis"
Show me the skill catalog
Install the "financial-report" skill
```

Installed skills with MCP endpoints return connection instructions so you can wire them up.

Connecting External MCP Servers

The Model Context Protocol (MCP) lets you connect external data sources to the agent at runtime. Each MCP server exposes its own set of tools that become available to the agent.

Connecting a Server

```
Connect to MCP server "my-data-server" at https://mcp.example.com/sse using SSE transport
```

The agent calls `connect_mcp_server`, establishes a connection, discovers the server's tools, and registers them in the dynamic tool registry. The tools become available immediately with the `mcp`:

prefix.

Supported transports:

- `sse` -- Server-Sent Events over HTTP (most common).
- `http` -- Standard HTTP transport.

Listing Connections

```
List my MCP connections
```

Shows all active MCP server connections and the tools each provides.

Disconnecting a Server

```
Disconnect the my-data-server MCP connection
```

Removes the connection and unregisters its tools.

How Tools Are Registered

AI Agent Canvas uses a dynamic tool registry that aggregates tools from multiple sources:

1. **Built-in tools** -- Compiled into the application (MarketData tools).
2. **Agent tools** -- Persona, workflow, entity, scheduling, guardrail, and profile management tools.
3. **Dynamic MCP tools** -- Registered at runtime when you connect an MCP server.

All tools are available to the agent simultaneously. The agent selects the appropriate tool based on your request.

Tips

- **Skills are LLM-powered** -- They run prompt templates, not code. This makes them easy to create but means they depend on the LLM's capabilities.
- **MCP connections persist during the session** but are not saved across backend restarts. Reconnect MCP servers after restarting.
- **Tool names must be unique.** If an MCP server provides a tool with the same name as a built-in tool, the MCP version is registered under the `mcp:server-name` namespace.

Scheduling

AI Agent Canvas includes a task scheduler powered by [Hangfire](#) that runs agent tasks in the background on a schedule or after a delay.

What Scheduled Tasks Do

A scheduled task runs an agent prompt at a specified time or on a recurring schedule. The agent executes the prompt as if you had typed it in the chat, including access to all tools, personas, and context. Results are stored and can be retrieved later.

Use cases include:

- Periodic market data summaries
- Recurring report generation
- Timed alerts and checks
- Batch data processing

Creating a Recurring Task

Ask the agent to create a recurring task using natural language:

Schedule a recurring task every hour to get stock quotes for AAPL and MSFT and summarize the price movement

The agent calls `schedule_recurring_task` with a cron expression derived from your request. Common schedules:

Request	Cron Expression
Every hour	0 * * * *
Every day at 9 AM	0 9 * * *
Every Monday at 8 AM	0 8 * * 1
Every 30 minutes	*/30 * * * *
Every weekday at 6 PM	0 18 * * 1-5

You can also specify a cron expression directly:

Schedule a recurring task with cron "0 9 * * 1-5" to generate a morning market briefing

Creating a One-Time Task

For tasks that should run once after a delay:

Schedule a one-time task in 30 minutes to check the current price of GOOGL

The agent calls `schedule_one_time_task` with the specified delay. The task runs once and stores its result.

Viewing Scheduled Tasks

List all scheduled tasks:

```
List my scheduled tasks
```

The agent calls `list_scheduled_tasks` and returns the name, schedule, and status of each task.

Getting Task Results

After a task has run, retrieve its results:

```
Get the results of my scheduled tasks
```

The agent calls `get_task_results` and returns the output from completed task executions.

Removing a Scheduled Task

```
Remove the market briefing scheduled task
```

The agent calls `remove_scheduled_task` to cancel and delete the task.

How It Works

The scheduling system uses three components:

1. **SchedulerToolProvider** -- Exposes the five scheduling tools to the agent:
`schedule_recurring_task`, `schedule_one_time_task`, `list_scheduled_tasks`, `remove_scheduled_task`, and `get_task_results`.
2. **Hangfire** -- An open-source background job framework for .NET. It manages job queues, scheduling, retries, and persistence.
3. **SQLite storage** -- Task state is persisted in `hangfire.db`, so scheduled tasks survive backend restarts.

When a scheduled task fires, Hangfire invokes the agent with the stored prompt. The agent processes it using the same pipeline as a chat message, including tool calls and context injection.

The Hangfire Dashboard

Hangfire provides a built-in web dashboard for monitoring and managing background jobs. Access it at:

```
http://localhost:5000/hangfire
```

The dashboard shows:

- **Recurring jobs** -- All recurring tasks with their cron expressions and next run times.
- **Succeeded jobs** -- Completed task executions with timestamps.
- **Failed jobs** -- Tasks that encountered errors, with exception details.
- **Processing jobs** -- Currently running tasks.
- **Scheduled jobs** -- One-time tasks waiting to execute.

Use the dashboard to inspect task history, trigger manual reruns, or delete jobs directly.

Warning: The Hangfire dashboard is exposed without authentication by default. In production, configure authentication middleware to restrict access. See the [Hangfire documentation](#) for details.

Tips

- **Tasks run server-side** -- They execute even when no browser is open.
- **Results are stored** -- You can retrieve results at any time after execution.
- **Tasks use the current agent configuration** -- The active persona, guardrails, and context at the time of execution apply to the task.
- **Monitor via notifications** -- Completed tasks can push results to the notification channel (`/api/notifications`), which the frontend receives via SSE.
- **SQLite persistence** -- The `hangfire.db` file stores all job state. Back it up if your scheduled tasks are important.

Workflows

Workflows let you define multi-step agent sequences that run as a single operation. Instead of typing each step manually, you create a workflow once and run it whenever you need it.

What Is a Workflow?

A workflow is a named sequence of steps stored as a markdown file. Each workflow has:

- **Name** -- A short identifier (e.g., "Morning Briefing").
- **Description** -- What the workflow accomplishes.
- **Tags** -- Optional labels for organization.
- **Content** -- The full workflow definition, including steps.

When you run a workflow, the agent reads its definition, builds a prompt from the steps, and executes it through the LLM. The agent has access to all its tools during execution, so workflow steps can fetch data, create entities, and perform any action the agent normally can.

Creating a Workflow

Ask the agent to create a workflow in natural language:

```
Create a workflow called "Morning Briefing" with description "Daily market overview" that does the following:  
1. Get stock quotes for AAPL, MSFT, GOOGL, and AMZN  
2. Get technical indicators for each stock  
3. Summarize the overall market sentiment  
4. List any stocks with significant price changes
```

The agent calls `create_workflow` and saves the workflow as a markdown file in `agent-data/workflows/`.

Listing Workflows

To see all available workflows:

```
List my workflows
```

The agent calls `list_workflows` and returns the name, description, and tags of each workflow.

Reading a Workflow

To view the full definition of a workflow:

```
Show me the Morning Briefing workflow
```

The agent calls `read_workflow` and displays the complete workflow content, including all steps.

Running a Workflow

To execute a workflow:

```
Run the Morning Briefing workflow
```

The agent calls `run_workflow`, which:

1. Loads the workflow definition from its markdown file.
2. Builds a prompt from the workflow content.
3. Executes the prompt through the LLM with full tool access.
4. Returns the combined results.

The agent executes the steps sequentially, using tool calls as needed. You see the results stream in as the workflow progresses.

Deleting a Workflow

```
Delete the Morning Briefing workflow
```

The agent calls `delete_workflow` and removes the markdown file.

Workflow File Format

Workflows are stored as markdown files in `agent-data/workflows/`. Each file contains YAML frontmatter followed by the workflow content:

```
---
name: Morning Briefing
description: Daily market overview
tags:
- market
- daily
---

## Steps

1. Get current stock quotes for AAPL, MSFT, GOOGL, and AMZN
2. Retrieve technical indicators for each stock
3. Summarize the overall market sentiment based on the data
4. Highlight any stocks with price changes greater than 2%
```

You can also edit these files directly if you prefer working with files over chat commands.

Example Workflows

Portfolio Review

```
Create a workflow called "Portfolio Review" that:
1. Gets stock quotes for my portfolio: AAPL, MSFT, GOOGL, AMZN, TSLA
2. Gets EDGAR company facts for any stock that moved more than 3%
3. Produces a summary table with ticker, price, change, and recommendation
```

Competitor Analysis

Create a workflow called "Competitor Analysis" that:

1. Gets stock quotes for CRM, NOW, WDAY, and HUBS
2. Gets technical indicators for each
3. Compares their relative strength
4. Produces a competitive landscape summary

End of Day Report

Create a workflow called "End of Day Report" that:

1. Gets stock quotes for the top 5 holdings
2. Summarizes the day's market movements
3. Lists any significant news or filings from EDGAR
4. Recommends actions for the next trading day

Combining Workflows with Scheduling

Workflows become more powerful when combined with scheduled tasks. For example:

Schedule a recurring task every weekday at 9 AM to run the Morning Briefing workflow

This creates a Hangfire job that executes your workflow automatically. See [Scheduling](#) for details.

Tips

- **Workflows are LLM-driven** -- Steps are instructions, not code. The LLM interprets and executes them using available tools.
- **Tool access is automatic** -- The agent can call any registered tool during workflow execution, including MCP connections.
- **Results depend on available data** -- If a tool call fails (e.g., API rate limit), the workflow continues with what it has and notes the failure.
- **Keep steps clear and specific** -- Vague steps produce inconsistent results. "Get the stock quote for AAPL" works better than "check the market."
- **Edit files directly** -- Workflow markdown files in `agent-data/workflows/` can be edited with any text editor for fine-tuning.

Security

AI Agent Canvas includes a layered security system covering governance policies, rate limiting, prompt injection detection, tool-call rules, security headers, and audit logging.

Governance Policy Overview

The governance policy defines rules that control what the agent can and cannot do. Rules are defined in a YAML file (default: `governance-policy.yaml` in the web project root) and enforced by the `GovernanceKernel` at runtime.

The default policy includes these rules:

Rule	Action	Description
block-dangerous-tools	Deny	Blocks execution of the <code>run_script</code> tool.
restrict-file-write-paths	Deny	Prevents <code>write_file</code> from targeting system directories (<code>/etc</code> , <code>/var</code> , <code>C:\Windows</code> , <code>C:\Program Files</code>).
block-private-mcp-endpoints	Deny	Blocks <code>connect_mcp_server</code> from connecting to private or internal network addresses (SSRF protection).
limit-scheduled-tasks	Deny	Blocks scheduling tools when the rate limit is exceeded.
allow-all-other	Allow	Permits all tool calls not matched by a deny rule.

Policy File Format

```
name: AiAgentCanvas-default
rules:
- name: block-dangerous-tools
  action: deny
  tools:
  - run_script
  description: Block script execution

- name: restrict-file-write-paths
  action: deny
  tools:
  - write_file
  paths:
  - /etc
  - /var
  - C:\Windows
  - C:\Program Files
  description: Restrict file writes to safe locations

- name: allow-all-other
  action: allow
  description: Permit everything else
```

Customizing the Policy

Edit `governance-policy.yaml` to add, modify, or remove rules. The policy path is configured in `appsettings.json`:

```
{
  "Security": {
    "PolicyPath": "governance-policy.yaml"
  }
}
```

The conflict resolution strategy is **deny overrides** -- if any rule denies a tool call, it is blocked regardless of other allow rules.

Rate Limiting

The backend enforces a fixed-window rate limit on all API requests. When a client exceeds the limit, the server responds with **HTTP 429 Too Many Requests**.

Configure the limit in `appsettings.json`:

```
{
  "Security": {
    "RateLimitPerMinute": 30
  }
}
```

The default is 30 requests per minute. Increase this for high-throughput environments or decrease it to conserve API quota.

Prompt Injection Detection

The `GovernanceKernel` includes built-in prompt injection detection. When enabled, it scans incoming messages for patterns that attempt to override the agent's instructions or extract system prompts.

This feature is enabled by default (`EnablePromptInjectionDetection: true`). Suspicious messages are flagged in the audit log and may be blocked depending on the governance configuration.

Tool-Call Governance

Every tool call passes through the governance pipeline before execution. The pipeline evaluates the call against all active policy rules:

1. The agent decides to call a tool (e.g., `connect_mcp_server`).
2. The governance kernel evaluates the call against deny rules.
3. If any deny rule matches, the call is blocked and the agent receives an error.
4. If no deny rule matches, the call proceeds.

Governed MCP Gateway

MCP connections receive additional scrutiny through the GovernedMcpGateway:

- **Suspicious payload blocking** -- Payloads that contain potential injection patterns are blocked (BlockOnSuspiciousPayload: true).
- **Approval-required tools** -- Certain tools (run_script, write_file) require explicit approval before execution.

Guardrails (User-Defined Safety Rules)

In addition to the system-level governance policy, you can create user-defined guardrail rules through conversation. Guardrails are injected into the agent's system prompt and guide its behavior.

Creating a Guardrail

Create a guardrail called "No Investment Advice" with severity "critical" and rule "Never provide specific investment recommendations or buy/sell signals. Always include a disclaimer that this is not financial advice."

Managing Guardrails

List my guardrails
Toggle the "No Investment Advice" guardrail off
Update the "No Investment Advice" guardrail to also prohibit price predictions
Delete the "No Investment Advice" guardrail

Guardrail severities:

Severity	Purpose
critical	Rules the agent must never violate.
warning	Rules the agent should follow but may deviate from with justification.
info	Guidance that shapes behavior but does not restrict it.

Guardrails are stored as markdown files in agent-data/guardrails/ and injected into the system prompt by the GuardrailContextProvider.

Security Headers

The backend adds the following security headers to all HTTP responses:

Header	Value	Purpose
X-Content-Type-Options	nosniff	Prevents MIME-type sniffing.
X-Frame-Options	DENY	Prevents the page from being embedded in frames (clickjacking protection).

Referrer-Policy	strict-origin-when-cross-origin	Controls referrer information sent with requests.
-----------------	---------------------------------	---

These headers are applied automatically and require no configuration.

Audit Logging

The governance system logs all tool calls and policy evaluations. Audit logging is enabled by default (`EnableAudit: true`) and records:

- Every tool call made by the agent
- The governance rules evaluated
- Whether the call was allowed or denied
- Timestamps and request details

Audit logs appear in the standard .NET logging output. Configure the log level in `appsettings.json` to control verbosity:

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information"
    }
  }
}
```

Set to Debug for detailed governance evaluation traces.

Security Checklist for Production

- Replace the `AIFoundry.Key` with a Managed Identity (`UseAzureCredential: true`).
- Set the `AlphaVantage.ApiKey` to a production key (not "demo").
- Review and customize `governance-policy.yaml` for your use case.
- Add authentication to the Hangfire dashboard (`/hangfire`).
- Adjust `RateLimitPerMinute` for your expected load.
- Set `AllowedHosts` to your specific domain instead of `"*"`.
- Enable HTTPS and configure appropriate TLS settings.
- Review audit logs regularly for unusual tool call patterns.

Developer Guide

Architecture deep-dive, project structure, and how to extend the system with custom agents and MCP connections.

Architecture Overview

AI Agent Canvas is a .NET 9 multi-agent copilot framework built on Microsoft's Agent Framework (MAF) and Microsoft.Extensions.AI. It connects a CopilotKit frontend to Azure AI Foundry models through an AG-UI Server-Sent Events protocol, with dynamic tool registration, governance, and markdown-persisted agent data.

Request Flow

```
User (Browser)
|
v
CopilotKit Frontend (Next.js)
| POST /api/copilotkit (AG-UI protocol)
v
AgUiEndpoint (SSE)
| Extracts messages, creates ChatClientAgent session
v
ChatClientAgent (MAF)
| AIContextProviders inject: system prompt, persona,
| guardrails, entities, context, dynamic tools
v
Azure AI Foundry (LLM)
| Model decides which tools to call
v
Tool Execution
| DynamicToolRegistry resolves registered AITools
| (MarketData, Skills, MCP connections, AgentData CRUD, etc.)
v
Streaming Response
| SSE events: run.started -> text.message.start ->
| text.message.content (deltas) -> text.message.end -> run.finished
v
CopilotKit renders response
```

Framework vs Custom Separation

The solution enforces a strict boundary between framework code and business logic.

Framework projects (under `src/AiAgentCanvas.*`) provide the engine: orchestration, tool registry, agent data persistence, skills management, security, scheduling, and notifications. These never change per use case.

Custom projects (under `src/Custom/`) hold all business-specific code: agent prompts, MCP data connections, custom vector stores. You add your own projects here without modifying any framework code.

```
src/
AiAgentCanvas.Abstractions/ <-- Shared interfaces (no dependencies)
AiAgentCanvas.Core/ <-- Orchestration engine
AiAgentCanvas.AgentData/ <-- MD-persisted agent data (6 domains)
AiAgentCanvas.Skills/ <-- Skill store + MCP connections
```

```

AiAgentCanvas.Security/ <-- Governance + rate limiting
AiAgentCanvas.Scheduler/ <-- Hangfire scheduled tasks
AiAgentCanvas.Notifications/ <-- SSE notification channel
AiAgentCanvas.SystemTools/ <-- Optional file I/O + script tools
AiAgentCanvas.Web/ <-- Composition root (Program.cs)
Custom/
HelloWorldAgent/ <-- Sample agent (prompts only)
MCP.MarketData/ <-- SEC EDGAR + Alpha Vantage tools
VectorStore.Sqlite/ <-- SQLite vector store for RAG

```

Dependency Flow

All projects depend downward toward Abstractions. The Web project is the only one that references everything.

```

AiAgentCanvas.Web
/ | | \ \
/ | | \ \
Core AgentData Skills Security Scheduler Notifications
\ | | /
\ | | /
Abstractions (shared types)

Custom/MCP.MarketData ----> Microsoft.Extensions.AI
Custom/HelloWorldAgent ----> (no dependencies)

```

Key dependency rules:

- **Abstractions** has zero project references. It defines `MarkdownFile`, `DocumentRecord`, `INotificationSink`, and `AgentNotification`.
- **Core** references **Abstractions**. It provides `IChatClient` registration, `DynamicToolRegistry`, `AIContextProvider` base, and the AG-UI endpoint.
- **AgentData**, **Skills**, **Security**, **Scheduler**, **Notifications** each reference **Core** and/or **Abstractions**.
- **Custom projects** reference only `Microsoft.Extensions.AI` (for `AIFunctionFactory`) -- they do not reference **Core** or any framework project.
- **Web** references everything and wires it together in `Program.cs`.

Key Packages

Package	Version	Purpose
Microsoft.Agents.AI	1.10.0	ChatClientAgent, AIContextProvider, AIAgent
Microsoft.Extensions.AI	10.7.0	IChatClient, AITool, AIFunctionFactory, ChatOptions
ModelContextProtocol	1.4.0	McpClient, HttpClientTransport for MCP servers

Microsoft.AgentGovernance	4.0.0	GovernanceKernel, PolicyEngine, InjectionDetector
Azure.AI.Inference	1.0.0-beta.5	ChatCompletionsClient for Azure AI Foundry
Hangfire	1.8.23	Background job scheduling with SQLite storage

Extension Method Pattern

Every project exposes its DI registration through `*ServiceExtensions.cs` files with `Add*()` methods and optionally `Use*()` methods for middleware:

```
// Each project follows this pattern:
builder.Services.AddAiAgentCanvasSecurity(builder.Configuration); // Security first
builder.Services.AddAiAgentCanvas(builder.Configuration, options => { ... }); // Core
builder.Services.AddMarketDataTools(); // Custom tools
builder.Services.AddAiAgentCanvasNotifications();
builder.Services.AddAiAgentCanvasScheduler();
// ... more domain registrations

app.UseAiAgentCanvasSecurity(); // Security middleware
app.UseAiAgentCanvas(); // CORS + AG-UI endpoint + health check
app.MapNotificationEndpoints(); // SSE notification stream
```

Tools are registered as `IReadOnlyList<AITool>` singleton services. The Core framework collects all registered `IReadOnlyList<AITool>` services at startup and passes them to the `ChatClientAgent` via `ChatOptions.Tools`. Additionally, `DynamicToolRegistry` allows runtime tool registration (used by MCP connections).

Next Steps

- [Project Structure](#) -- detailed breakdown of every project
- [Core Framework](#) -- how orchestration, tools, and context work
- [Agent Data](#) -- the 6 markdown-persisted data domains
- [Skills and MCP](#) -- skill management and MCP connections
- [Security](#) -- governance policies and rate limiting
- [Adding Custom Agents](#) -- build your own agent
- [Adding Custom MCP Connections](#) -- add data sources

Project Structure

This page describes every project in the AI Agent Canvas solution, its purpose, key files, and dependencies.

Solution Layout

The solution is organized into two solution folders:

- **AiAgentCanvas** -- framework projects that ship as-is
- **Custom** -- business-specific projects you create and modify

```
AiAgentCanvas.sln
src/
AiAgentCanvas.Abstractions/ Shared types (zero dependencies)
AiAgentCanvas.Core/ Orchestration engine
AiAgentCanvas.AgentData/ Markdown-persisted agent data
AiAgentCanvas.Skills/ Skill store + MCP connections
AiAgentCanvas.Security/ Governance + rate limiting
AiAgentCanvas.Scheduler/ Hangfire scheduled tasks
AiAgentCanvas.Notifications/ In-memory notification sink + SSE
AiAgentCanvas.SystemTools/ Optional file I/O and script tools
AiAgentCanvas.Web/ Composition root
Custom/
HelloWorldAgent/ Sample agent prompts
MCP.MarketData/ SEC EDGAR + Alpha Vantage tools
VectorStore.Sqlite/ SQLite vector store for RAG
```

Framework Projects

AiAgentCanvas.Abstractions

Shared interfaces and utility types with zero project references. Every other project may reference this.

File	Purpose
MarkdownFile.cs	YAML frontmatter + body parsing, serialization, and file loading
DocumentRecord.cs	Shared document record type
INotificationSink.cs	Interface for notification delivery

MarkdownFile is the foundation of agent data persistence. It parses files with YAML frontmatter delimited by --- and provides Parse(), Write(), LoadAll(), and SanitizeFileName() methods.

AiAgentCanvas.Core

The orchestration engine. Registers the LLM client, tool registry, context providers, and the AG-UI SSE endpoint.

File	Purpose
ServiceCollectionExtensions.cs	AddAiAgentCanvas() and UseAiAgentCanvas()
Endpoints/AgUiEndpoint.cs	POST /api/copilotkit -- AG-UI SSE streaming
Services/AIFoundryClientFactory.cs	Creates ChatCompletionsClient from config
Skills/DynamicToolRegistry.cs	Runtime tool registration by source key
Skills/DynamicToolContextProvider.cs	Injects dynamic tools into ChatOptions
Providers/RagContextProvider.cs	RAG context injection from vector store
Configuration/AIFoundryOptions.cs	Options: Endpoint, Key, DeploymentName, etc.

Dependencies: Abstractions, Microsoft.Agents.AI, Microsoft.Extensions.AI, Azure.AI.Inference

AiAgentCanvas.AgentData

Six markdown-persisted data domains. Each domain follows an identical pattern: Store + ToolProvider + AIContextProvider.

Subdirectory	Domain	Storage Path
Personas/	Agent personas with custom instructions	agent-data/personas/*.md
Context/	Persistent context entries	agent-data/context/*.md
Workflows/	Multi-step workflows	agent-data/workflows/*.md
Guardrails/	Behavioral guardrails	agent-data/guardrails/*.md
Entities/	Domain entities and schemas	agent-data/entities/*.md
Profiles/	User profiles	agent-data/profiles/*.md

File	Purpose
AgentDataServiceExtensions.cs	Six Add*() methods, one per domain

Dependencies: Core, Abstractions, Microsoft.Agents.AI, Microsoft.Extensions.AI

AiAgentCanvas.Skills

Skill persistence, MCP connection management, and skill authoring tools.

File	Purpose
SkillsServiceExtensions.cs	AddAiAgentCanvasSkills(), AddAiAgentCanvasMcp(), AddAiAgentCanvasSkillRegistry(), AddAiAgentCanvasSkillAuthoring()
SkillStore.cs	SQLite-backed skill persistence
McpConnectionManager.cs	Connects to external MCP servers, registers tools
LocalSkillRegistry.cs	Resolves skills from local files at runtime
SkillAuthoringToolProvider.cs	Tools for creating/editing skills via chat
SkillToolProvider.cs	Tools for managing stored skills

Dependencies: Core, ModelContextProtocol, Microsoft.Extensions.AI

AiAgentCanvas.Security

Microsoft.AgentGovernance integration, prompt injection detection, MCP gateway, and ASP.NET rate limiting.

File	Purpose
SecurityServiceExtensions.cs	AddAiAgentCanvasSecurity() and UseAiAgentCanvasSecurity()
GovernanceContextProvider.cs	Scans instructions for prompt injection before LLM calls
GovernedMcpGateway.cs	Wraps McpGateway for tool allow/deny decisions

Dependencies: AgentGovernance, Microsoft.Agents.AI, ASP.NET rate limiting

AiAgentCanvas.Scheduler

Hangfire-based scheduled task management with SQLite storage.

File	Purpose
SchedulerServiceExtensions.cs	AddAiAgentCanvasScheduler()

Dependencies: Hangfire, Hangfire.Storage.SQLite

AiAgentCanvas.Notifications

In-memory notification channel using System.Threading.Channels with an SSE delivery endpoint.

File	Purpose
NotificationServiceExtensions.cs	AddAiAgentCanvasNotifications()

Dependencies: Abstractions

AiAgentCanvas.SystemTools

Optional file I/O and script execution tools. These are governed by the security layer.

File	Purpose
SystemToolsServiceExtensions.cs	AddAiAgentCanvasSystemTools()

Dependencies: Core, Microsoft.Extensions.AI

AiAgentCanvas.Web

The composition root. Contains only Program.cs and configuration files. This is the only project that references all other projects.

File	Purpose
------	---------

Program.cs	Wires all services and middleware
appsettings.json	AI Foundry config, security settings
governance-policy.yaml	Default governance rules

Custom Projects

The `src/Custom/` folder is where you add all business-specific code. Each custom project is a standalone class library that gets wired into the composition root.

Custom/HelloWorldAgent

A minimal sample agent containing only a prompts class. Demonstrates that an agent can be pure reasoning logic with zero framework dependencies.

File	Purpose
HelloWorldAgent.csproj	net9.0, no package references
HelloWorldAgentPrompts.cs	GetSystemPrompt() returns the agent's system prompt

Custom/MCP.MarketData

A data connection providing SEC EDGAR and Alpha Vantage tools. Demonstrates the ToolProvider + ServiceExtensions pattern.

File	Purpose
MarketDataToolProvider.cs	Three AI Tools: edgar_company_facts, stock_quote, stock_technicals
MarketDataServiceExtensions.cs	AddMarketDataTools() registers HttpClient and tools

Dependencies: Microsoft.Extensions.AI, Microsoft.Extensions.Http.Resilience

Custom/VectorStore.Sqlite

A SQLite-based vector store used for RAG (Retrieval-Augmented Generation).

Adding Your Own Custom Projects

1. Create a new folder under `src/Custom/`
2. Add a `.csproj` targeting `net9.0`
3. Implement your agent prompts or tool providers
4. Add a `ProjectReference` in `AiAgentCanvas.Web.csproj`
5. Add the project to `AiAgentCanvas.sln` under the Custom solution folder
6. Wire up in `Program.cs`

See [Adding Custom Agents](#) and [Adding Custom MCP Connections](#) for detailed walkthroughs.

Core Framework

The `AiAgentCanvas.Core` project is the orchestration engine. It registers the LLM client, manages tools and context providers, and exposes the AG-UI SSE endpoint that CopilotKit connects to.

ServiceCollectionExtensions

The `AddAiAgentCanvas()` extension method is the central registration point. It wires up the chat client, tool registry, context providers, and the `ChatClientAgent`.

```
builder.Services.AddAiAgentCanvas(builder.Configuration, options =>
{
    options.AgentName = "AiAgentCanvas";
    options.AgentDescription = "A multi-tool AI assistant with market data, "
+ "scheduling, skills, and MCP integration.";
});
```

What AddAiAgentCanvas() Registers

1. **AIFoundryOptions** -- binds the AIFoundry configuration section
2. **AIFoundryClientFactory** -- singleton that creates `ChatCompletionsClient`
3. **IChatClient** -- via `AddChatClient()`, resolved from the factory
4. **DynamicToolRegistry** -- singleton for runtime tool registration
5. **DefaultSystemPrompt** -- wraps the configured or default system prompt
6. **SystemPromptProvider** -- `AIContextProvider` that sets initial instructions
7. **DynamicToolContextProvider** -- `AIContextProvider` that injects dynamic tools
8. **ChatClientAgent** -- the MAF agent with all tools and context providers
9. **CORS** -- allows any origin (intended for local CopilotKit development)

The agent is constructed with all `IReadOnlyList<AITool>` services flattened into `ChatOptions.Tools`, and all `AIContextProvider` services passed as the context pipeline:

```
var tools = sp.GetServices<IReadOnlyList<AITool>>().SelectMany(t => t).ToList();

var agentOptions = new ChatClientAgentOptions
{
    Name = options.AgentName,
    Description = options.AgentDescription,
    ChatOptions = new ChatOptions { Tools = tools },
    AIContextProviders = contextProviders,
};

var agent = new ChatClientAgent(chatClient, agentOptions, loggerFactory, sp);
```

AiAgentCanvasOptions

```
public sealed class AiAgentCanvasOptions
{
    public string AgentName { get; set; } = "AiAgentCanvas";
    public string AgentDescription { get; set; } = "A multi-tool AI assistant";
}
```

```
public string? SystemPrompt { get; set; }
}
```

UseAiAgentCanvas()

The UseAiAgentCanvas() method configures the middleware pipeline:

```
public static WebApplication UseAiAgentCanvas(this WebApplication app)
{
    app.UseCors();
    app.MapAgUiEndpoints();
    app.MapGet("/api/health", () => Results.Ok(new { status = "healthy" }));
    return app;
}
```

AIFoundryClientFactory

Creates the Azure AI Foundry chat client from configuration. Supports both API key and Azure credential authentication.

```
public IChatClient CreateChatClient()
{
    var endpoint = new Uri(_options.Endpoint);
    var inner = _options.UseAzureCredential
        ? new ChatCompletionsClient(endpoint, new DefaultAzureCredential())
        : new ChatCompletionsClient(endpoint, new AzureKeyCredential(_options.Key!));

    return inner.AsIChatClient(_options.DeploymentName);
}
```

AIFoundryOptions

Configured in appsettings.json under the AIFoundry section:

```
{
  "AIFoundry": {
    "Endpoint": "https://your-resource.services.ai.azure.com/models",
    "Key": "your-api-key",
    "DeploymentName": "gpt-4o",
    "EmbeddingDeploymentName": "text-embedding-3-small",
    "UseAzureCredential": false
  }
}
```

Property	Required	Description
Endpoint	Yes	Azure AI Foundry endpoint URL
Key	Conditional	API key (required when UseAzureCredential is false)
DeploymentName	Yes	Model deployment name
EmbeddingDeploymentName	No	Embedding model for RAG
UseAzureCredential	No	Use DefaultAzureCredential instead of API key

AgUiEndpoint

The AG-UI endpoint handles the CopilotKit frontend protocol. It accepts POST requests at `/api/copilotkit`, extracts chat messages, runs the agent, and streams responses as SSE events.

```
app.MapPost("/api/copilotkit", HandleCopilotKitRequest);
```

Request Format

CopilotKit sends a JSON body with messages and an optional `threadId`:

```
{
  "threadId": "abc-123",
  "messages": [
    { "role": "user", "content": "What is Apple's latest revenue?" }
  ]
}
```

SSE Event Sequence

The endpoint streams events in this order:

```
event: run.started
data: {"threadId":"abc-123","runId":"def-456"}

event: text.message.start
data: {"messageId":"ghi-789","role":"assistant","agentName":"AiAgentCanvas"}

event: text.message.content
data: {"messageId":"ghi-789","delta":"Apple's latest"}

event: text.message.content
data: {"messageId":"ghi-789","delta":" quarterly revenue was..."}

event: text.message.end
data: {"messageId":"ghi-789"}

event: run.finished
data: {"threadId":"abc-123","runId":"def-456"}
```

How It Works

1. Deserializes the request body and extracts messages
2. Maps each message to a `ChatMessage` with the appropriate `ChatRole`
3. Creates a new agent session via `agent.CreateSessionAsync()`
4. Sets the response content type to `text/event-stream`
5. Emits `run.started` and `text.message.start` events
6. Iterates `agent.RunStreamingAsync()` and emits `text.message.content` for each text delta
7. Emits `text.message.end` and `run.finished`

DynamicToolRegistry

A thread-safe registry that allows tools to be added and removed at runtime. Used primarily by `McpConnectionManager` to register tools from external MCP servers.

```
public sealed class DynamicToolRegistry
{
    private readonly ConcurrentDictionary<string, List<AITool>> _toolsBySource = new();

    public void Register(string source, IEnumerable<AITool> tools)
    {
        _toolsBySource[source] = tools.ToList();
    }

    public void Unregister(string source)
    {
        _toolsBySource.TryRemove(source, out _);
    }

    public IReadOnlyList<AITool> GetAllTools()
    {
        return _toolsBySource.Values.SelectMany(t => t).ToList();
    }
}
```

Tools are keyed by a source string (e.g., "mcp:my-server"). When an MCP server connects, its tools are registered under that key. When it disconnects, they are removed.

DynamicToolContextProvider

An `AIContextProvider` that merges dynamic tools into the chat options before each LLM call. This ensures tools registered at runtime (via MCP connections) are available to the model.

```
internal sealed class DynamicToolContextProvider : AIContextProvider
{
    private readonly DynamicToolRegistry _registry;

    protected override ValueTask<AIContext> ProvideAIContextAsync(
        InvokingContext context, CancellationToken cancellationToken)
    {
        var dynamicTools = _registry.GetAllTools();
        if (dynamicTools.Count > 0)
        {
            var existing = context.AIContext.Tools?.ToList() ?? [];
            existing.AddRange(dynamicTools);
            context.AIContext.Tools = existing;
        }
        return new ValueTask<AIContext>(context.AIContext);
    }
}
```

Tool Registration: Two Paths

Tools reach the agent through two mechanisms:

1. **Startup registration** -- Tool providers register `IReadOnlyList<AITool>` as singleton services. These are collected once at startup and passed to `ChatClientAgent` via `ChatOptions.Tools`.

2. **Runtime registration** -- Tools registered into `DynamicToolRegistry` are injected by `DynamicToolContextProvider` before each LLM call. This is used for MCP connections that can come and go.

Both paths result in the LLM seeing all available tools in its context window.

Context Provider Pipeline

`AIContextProvider` subclasses form a pipeline that runs before each LLM invocation. Each provider can modify the `AIContext` (instructions, tools, or other metadata). The framework registers several providers:

Provider	Source	Purpose
<code>SystemPromptProvider</code>	Core	Sets the default system prompt
<code>DynamicToolContextProvider</code>	Core	Injects runtime-registered tools
<code>PersonaContextProvider</code>	AgentData	Appends active persona instructions
<code>GuardrailContextProvider</code>	AgentData	Appends guardrail constraints
<code>EntityContextProvider</code>	AgentData	Appends entity context
<code>PersistentContextProvider</code>	AgentData	Appends persistent context entries
<code>UserProfileContextProvider</code>	AgentData	Appends user profile context
<code>GovernanceContextProvider</code>	Security	Scans for prompt injection
<code>RagContextProvider</code>	Core	Injects RAG search results

Agent Data

The `AiAgentCanvas.AgentData` project provides six markdown-persisted data domains. Each domain follows an identical three-class pattern and gives the LLM tools to manage its own configuration at runtime.

The Six Domains

Domain	Storage Path	Purpose
Personas	agent-data/personas/*.md	Custom agent identities with instructions
Context	agent-data/context/*.md	Persistent context entries
Workflows	agent-data/workflows/*.md	Multi-step workflow definitions
Guardrails	agent-data/guardrails/*.md	Behavioral constraints
Entities	agent-data/entities/*.md	Domain entity schemas
UserProfiles	agent-data/profiles/*.md	User preference profiles

The Store / ToolProvider / ContextProvider Pattern

Every domain implements exactly three classes:

1. **Store** -- CRUD operations on markdown files in a directory
2. **ToolProvider** -- Exposes store operations as `AITool` instances the LLM can call
3. **AiContextProvider** (optional) -- Injects domain data into the system prompt before each LLM call

This pattern means the LLM can create, read, update, and delete domain data through tool calls, and that data automatically influences the LLM's behavior through context injection.

MarkdownFile: The Persistence Foundation

All agent data is stored as markdown files with YAML frontmatter. The `MarkdownFile` utility class in `AiAgentCanvas.Abstractions` handles parsing and serialization.

File Format

```
---
name: code-reviewer
description: Reviews code for quality and correctness
---

You are an expert code reviewer. When reviewing code:
- Check for bugs, security issues, and performance problems
- Suggest improvements with specific code examples
- Be constructive and explain your reasoning
```

Key Methods

```
// Parse a single file
MarkdownFile? file = MarkdownFile.Parse("agent-data/personas/code-reviewer.md");
string name = file.Get("name"); // "code-reviewer"
string desc = file.Get("description"); // "Reviews code for quality..."
string body = file.Body; // The instruction text

// Write a file (creates directory if needed)
MarkdownFile.Write(
    "agent-data/personas/code-reviewer.md",
    new Dictionary<string, string>
    {
        ["name"] = "code-reviewer",
        ["description"] = "Reviews code for quality and correctness",
    },
    "You are an expert code reviewer...");

// Load all files from a directory
List<MarkdownFile> all = MarkdownFile.LoadAll("agent-data/personas/");

// Sanitize a name for use as a filename
string safe = MarkdownFile.SanitizeFileName("Code Reviewer"); // "code-reviewer"
```

Detailed Walkthrough: Personas

Personas are the most illustrative domain. Here is how all three classes work together.

PersonaStore

The store manages CRUD operations on persona markdown files and tracks which persona is active via a `.active` marker file.

```
public sealed class PersonaStore
{
    private readonly string _directory;

    public PersonaStore(string directory) { /* creates directory if missing */ }

    public void SavePersona(string name, string description, string instructions)
    {
        MarkdownFile.Write(
            Path.Combine(_directory, MarkdownFile.SanitizeFileName(name) + ".md"),
            new Dictionary<string, string>
            {
                ["name"] = name,
                ["description"] = description,
            },
            instructions);
    }

    public List<PersonaInfo> ListPersonas()
    {
        return MarkdownFile.LoadAll(_directory)
            .Select(ToPersona)
            .Where(p => p is not null)
            .Cast<PersonaInfo>()
            .ToList();
    }
}
```



```

public PersonaInfo? GetPersona(string name) { ... }
public bool DeletePersona(string name) { ... }
public string? GetActivePersonaName() { ... }
public void SetActivePersona(string? name) { ... }
public string? GetActiveInstructions() { ... }
}

```

PersonaToolProvider

The tool provider creates `AITool` instances using `AIFunctionFactory.Create()`. Each tool delegates to the store.

```

public sealed class PersonaToolProvider
{
    private readonly PersonaStore _store;

    public IReadOnlyList<AITool> GetTools()
    {
        return
        [
            AIFunctionFactory.Create(CreatePersona, "create_persona",
                "Create a new persona with custom instructions"),
            AIFunctionFactory.Create(ListPersonas, "list_personas",
                "List all available personas"),
            AIFunctionFactory.Create(SwitchPersona, "switch_persona",
                "Switch to a different persona"),
            AIFunctionFactory.Create(ReadPersona, "read_persona",
                "Read the full details of a persona"),
            AIFunctionFactory.Create(UpdatePersona, "update_persona",
                "Update an existing persona"),
            AIFunctionFactory.Create(DeletePersona, "delete_persona",
                "Delete a persona"),
        ];
    }

    private string CreatePersona(string name, string description, string instructions)
    {
        var existing = _store.GetPersona(name);
        if (existing is not null)
            return JsonSerializer.Serialize(new { error = "Persona already exists..." });

        _store.SavePersona(name, description, instructions);
        return JsonSerializer.Serialize(new { status = "created", name });
    }

    // ... similar implementations for other tools
}

```

PersonaContextProvider

The context provider reads the active persona and appends its instructions to the system prompt before each LLM call.

```

internal sealed class PersonaContextProvider : AIContextProvider
{
    private readonly PersonaStore _store;
    private readonly string _defaultPrompt;
}

```

```
protected override ValueTask<AIContext> ProvideAIContextAsync(
    InvokingContext context, CancellationToken cancellationToken)
{
    var activeInstructions = _store.GetActiveInstructions();
    if (!string.IsNullOrEmpty(activeInstructions))
    {
        context.AIContext.Instructions =
            (context.AIContext.Instructions ?? "") + "\n" + activeInstructions;
    }
    else if (string.IsNullOrEmpty(context.AIContext.Instructions))
    {
        context.AIContext.Instructions = _defaultPrompt;
    }
    return new ValueTask<AIContext>(context.AIContext);
}
}
```

Registration

The `AgentDataServiceExtensions` class provides one `Add*( )` method per domain. Each method registers the Store, ToolProvider, tools, and (where applicable) the `AIContextProvider`.

```
public static IServiceCollection AddAiAgentCanvasPersonas(
    this IServiceCollection services,
    string directory = "./agent-data/personas")
{
    services.AddSingleton(new PersonaStore(directory));
    services.AddSingleton<PersonaToolProvider>();
    services.AddSingleton<IReadOnlyList<AITool>>(sp =>
        sp.GetRequiredService<PersonaToolProvider>().GetTools());
    services.AddSingleton<AIContextProvider>(sp =>
    {
        var store = sp.GetRequiredService<PersonaStore>();
        var defaultPrompt = sp.GetRequiredService<DefaultSystemPrompt>().Value;
        return new PersonaContextProvider(store, defaultPrompt);
    });
    return services;
}
```

All six domains are registered individually in `Program.cs`:

```
builder.Services.AddAiAgentCanvasPersonas();
builder.Services.AddAiAgentCanvasContext();
builder.Services.AddAiAgentCanvasWorkflows();
builder.Services.AddAiAgentCanvasGuardrails();
builder.Services.AddAiAgentCanvasEntities();
builder.Services.AddAiAgentCanvasUserProfiles();
```

Other Domains

Each domain follows the same pattern as `Personas` but with domain-specific data:

Domain	Context Provider	What it injects
Personas	PersonaContextProvider	Active persona instructions
Context	PersistentContextProvider	All persistent context entries

Guardrails	GuardrailContextProvider	Active guardrail constraints
Entities	EntityContextProvider	Entity schemas and definitions
UserProfiles	UserProfileContextProvider	Active user preferences
Workflows	(none)	Executed on demand via WorkflowExecutor

Creating a New Domain

To add a seventh agent data domain, follow the same pattern:

1. Create a subdirectory under `AiAgentCanvas.AgentData/` (e.g., `Templates/`)
2. Create a `TemplateStore` class with CRUD methods using `MarkdownFile`
3. Create a `TemplateToolProvider` class exposing tools via `AIFunctionFactory.Create()`
4. Optionally create a `TemplateContextProvider` extending `AIContextProvider`
5. Add an `AddAiAgentCanvasTemplates()` method to `AgentDataServiceExtensions`
6. Call the new method in `Program.cs`

Skills & MCP

The `AiAgentCanvas.Skills` project manages skill persistence, external MCP server connections, local skill resolution, and skill authoring. It enables the agent to connect to arbitrary MCP servers at runtime and register their tools dynamically.

Overview

The skills system has four components, each with its own registration method:

Component	Registration	Purpose
SkillStore + SkillToolProvider	<code>AddAiAgentCanvasSkills()</code>	SQLite-backed skill persistence and management
McpConnectionManager	<code>AddAiAgentCanvasMcp()</code>	Connect/disconnect external MCP servers at runtime
LocalSkillRegistry	<code>AddAiAgentCanvasSkillRegistry()</code>	Resolve skills from local markdown files
SkillAuthoringToolProvider	<code>AddAiAgentCanvasSkillAuthoring()</code>	Create and edit skills via chat

```
builder.Services.AddAiAgentCanvasSkills();
builder.Services.AddAiAgentCanvasMcp();
builder.Services.AddAiAgentCanvasSkillRegistry();
builder.Services.AddAiAgentCanvasSkillAuthoring();
```

SkillStore

The `SkillStore` persists skill definitions in a SQLite database. It provides CRUD operations for skill records.

```
public sealed class SkillStore
{
    public SkillStore(string connectionString = "Data Source=skills.db") { ... }
}
```

The `SkillToolProvider` exposes these operations as `AITools` so the LLM can manage skills through conversation.

Registration

```
public static IServiceCollection AddAiAgentCanvasSkills(
    this IServiceCollection services,
    string connectionString = "Data Source=skills.db")
{
    services.AddSingleton(new SkillStore(connectionString));
    services.AddSingleton<SkillToolProvider>();
    services.AddSingleton<IReadOnlyList<AITool>>(sp =>
        sp.GetRequiredService<SkillToolProvider>().GetTools());
    return services;
}
```

McpConnectionManager

The `McpConnectionManager` is the runtime MCP client. It allows the agent to connect to external MCP servers, discover their tools, and register those tools into the `DynamicToolRegistry` so the LLM can use them.

How It Works

1. The LLM calls the `connect_mcp_server` tool with a name, endpoint URL, and transport type
2. `McpConnectionManager` creates an `HttpClientTransport` and connects via `McpClient.CreateAsync()`
3. It calls `ListToolsAsync()` to discover the server's tools
4. The discovered tools are registered into `DynamicToolRegistry` under the key `mcp:{name}`
5. `DynamicToolContextProvider` picks up the new tools on the next LLM invocation

Exposed Tools

The manager itself exposes three management tools:

```
public IReadOnlyList<AITool> GetTools()
{
    return
    [
        AIFunctionFactory.Create(ConnectMcpServer, "connect_mcp_server",
            "Connect to an MCP server and register its tools"),
        AIFunctionFactory.Create(DisconnectMcpServer, "disconnect_mcp_server",
            "Disconnect from an MCP server and remove its tools"),
        AIFunctionFactory.Create(ListMcpConnections, "list_mcp_connections",
            "List all active MCP server connections"),
    ];
}
```

connect_mcp_server

Connects to an MCP server and registers its tools:

```
private async Task<string> ConnectMcpServer(string name, string endpoint,
    string transport, CancellationToken ct)
{
    IClientTransport clientTransport = transport.ToLowerInvariant() switch
    {
        "http" or "sse" => new HttpClientTransport(
            new HttpClientTransportOptions { Endpoint = new Uri(endpoint) }),
        _ => throw new ArgumentException($"Unsupported transport: {transport}"),
    };

    var client = await McpClient.CreateAsync(clientTransport, cancellationTokens: ct);
    var mcpTools = await client.ListToolsAsync(cancellationTokens: ct);
    var aiTools = mcpTools.Cast<AITool>().ToList();

    _connections[name] = new McpConnection { /* ... */ };
    _registry.Register($"mcp:{name}", aiTools);

    return JsonSerializer.Serialize(new {
```

```

status = "connected", name, endpoint,
toolCount = aiTools.Count,
tools = aiTools.Select(t => t.Name).ToList()
});
}

```

disconnect_mcp_server

Removes a connection and unregisters its tools from the DynamicToolRegistry:

```

private async Task<string> DisconnectMcpServer(string name, CancellationToken ct)
{
    if (!_connections.TryRemove(name, out var connection))
        return JsonSerializer.Serialize(new { error = $"Connection '{name}' not found" });

    _registry.Unregister($"mcp:{name}");
    if (connection.Client is not null)
        await connection.Client.DisposeAsync();

    return JsonSerializer.Serialize(new { status = "disconnected", name });
}

```

list_mcp_connections

Returns all active connections with their tool counts:

```

private string ListMcpConnections()
{
    var connections = _connections.Values.Select(c => new
    {
        c.Name, c.Endpoint, c.Transport,
        toolCount = c.Tools?.Count ?? 0,
    }).ToList();

    return JsonSerializer.Serialize(new { count = connections.Count, connections });
}

```

Registration

```

public static IServiceCollection AddAiAgentCanvasMcp(this IServiceCollection services)
{
    services.AddSingleton<McpConnectionManager>();
    services.AddSingleton<IReadOnlyList<AITool>>(sp =>
        sp.GetRequiredService<McpConnectionManager>().GetTools());
    return services;
}

```

LocalSkillRegistry

The LocalSkillRegistry resolves skills from local markdown files at runtime. It combines data from the SkillStore, McpConnectionManager, and skill files on disk.

```

public static IServiceCollection AddAiAgentCanvasSkillRegistry(
    this IServiceCollection services,
    string skillsDirectory = "./agent-data/skills")
{
    services.AddSingleton(sp => new LocalSkillRegistry(

```

```
skillsDirectory,
sp.GetRequiredService<SkillStore>(),
sp.GetRequiredService<McpConnectionManager>(),
sp.GetRequiredService<ILogger<LocalSkillRegistry>>());
services.AddSingleton<IReadOnlyList<AITool>>(sp =>
sp.GetRequiredService<LocalSkillRegistry>().GetTools());
return services;
}
```

SkillAuthoringToolProvider

Provides tools for creating and editing skill definitions through conversation. Skills are stored as markdown files in the skills directory.

```
public static IServiceCollection AddAiAgentCanvasSkillAuthoring(
this IServiceCollection services,
string skillsDirectory = "./agent-data/skills")
{
services.AddSingleton(sp => new SkillAuthoringToolProvider(
skillsDirectory,
sp.GetRequiredService<ILogger<SkillAuthoringToolProvider>>());
services.AddSingleton<IReadOnlyList<AITool>>(sp =>
sp.GetRequiredService<SkillAuthoringToolProvider>().GetTools());
return services;
}
```

MCP Connection Lifecycle

```
User: "Connect to my data server at https://data.example.com/mcp"
|
v
LLM calls connect_mcp_server(name="data", endpoint="https://...", transport="http")
|
v
McpConnectionManager:
1. Creates HttpClientTransport
2. Connects via McpClient.CreateAsync()
3. Lists tools via ListToolsAsync()
4. Registers tools: _registry.Register("mcp:data", aiTools)
|
v
Next LLM invocation:
DynamicToolContextProvider reads DynamicToolRegistry
-> MCP server's tools appear in ChatOptions.Tools
-> LLM can now call them
|
v
User: "Disconnect from the data server"
|
v
LLM calls disconnect_mcp_server(name="data")
|
v
McpConnectionManager:
1. _registry.Unregister("mcp:data")
2. Disposes McpClient
-> Tools removed from future invocations
```

Security Integration

MCP connections are subject to governance policies. The `GovernedMcpGateway` in the Security project can block connections to private/internal addresses (SSRF protection) and require approval for specific tools. See the [Security](#) page for details on the governance policy rules that apply to `connect_mcp_server`.

Security

The `AiAgentCanvas.Security` project integrates `Microsoft.AgentGovernance 4.0.0` for policy-based tool governance, prompt injection detection, and MCP gateway control. It also configures ASP.NET rate limiting and security headers.

Overview

Security is registered first in `Program.cs`, before any other service, so that governance is in place before tools are registered:

```
builder.Services.AddAiAgentCanvasSecurity(builder.Configuration);  
// ... all other registrations ...  
app.UseAiAgentCanvasSecurity();
```

What Gets Registered

Component	Purpose
GovernanceKernel	Central governance object with policy engine and audit
PolicyEngine	Evaluates YAML rules against tool calls
AuditEmitter	Logs all governance events
GovernanceContextProvider	Scans for prompt injection before each LLM call
GovernedMcpGateway	Wraps MCP tool calls for allow/deny decisions
Rate limiter	Fixed-window rate limiting on the <code>/api/copilotkit</code> endpoint
Security headers	X-Content-Type-Options, X-Frame-Options, Referrer-Policy

AddAiAgentCanvasSecurity()

The registration method accepts optional configuration callbacks for governance options and MCP gateway config:

```
public static IServiceCollection AddAiAgentCanvasSecurity(  
    this IServiceCollection services,  
    IConfiguration configuration,  
    Action<GovernanceOptions>? configureGovernance = null,  
    Action<McpGatewayConfig>? configureMcp = null)  
{  
    var governanceOptions = new GovernanceOptions  
    {  
        EnableAudit = true,  
        EnableMetrics = true,  
        EnablePromptInjectionDetection = true,  
        ConflictStrategy = ConflictResolutionStrategy.DenyOverrides,  
        PolicyPaths = policyPaths,  
    };  
  
    configureGovernance?.Invoke(governanceOptions);  
}
```

```

var kernel = new GovernanceKernel(governanceOptions);
services.AddSingleton(kernel);
services.AddSingleton(kernel.PolicyEngine);
services.AddSingleton(kernel.AuditEmitter);
// ...
}

```

GovernanceOptions Defaults

Option	Default	Description
EnableAudit	true	Log all governance decisions
EnableMetrics	true	Collect governance metrics
EnablePromptInjectionDetection	true	Scan system instructions for injection
ConflictStrategy	DenyOverrides	When rules conflict, deny wins
PolicyPaths	From config	Path to governance-policy.yaml

GovernanceKernel

The `GovernanceKernel` is the central governance object. It is constructed from `GovernanceOptions` and provides access to the `PolicyEngine`, `AuditEmitter`, and `InjectionDetector`.

During startup, `UseAiAgentCanvasSecurity()` subscribes to all governance events for logging:

```

kernel.OnAllEvents(e =>
{
    logger.LogInformation("[GOVERNANCE:AUDIT] Type={Type} Agent={Agent} Policy={Policy}",
        e.Type, e.AgentId, e.PolicyName ?? "none");
});

```

GovernanceContextProvider

An `AIContextProvider` that runs before each LLM call to scan system instructions for prompt injection attempts.

```

public sealed class GovernanceContextProvider : AIContextProvider
{
    private readonly GovernanceKernel _kernel;

    protected override ValueTask<AIContext> ProvideAIContextAsync(
        InvokingContext context, CancellationToken cancellation)
    {
        if (_kernel.InjectionDetector is null ||
            string.IsNullOrEmpty(context.AIContext.Instructions))
            return new ValueTask<AIContext>(context.AIContext);

        var result = _kernel.InjectionDetector.Detect(context.AIContext.Instructions);
        if (result.IsInjection)
        {
            _logger.LogWarning("[GOVERNANCE] Prompt injection detected: {Type}",
                result.InjectionType);
        }
    }
}

```

```

_kernel.AuditEmitter.Emit(
    GovernanceEventType.PolicyViolation,
    "system", "instructions",
    new Dictionary<string, object>
    {
        ["injectionType"] = result.InjectionType.ToString(),
        ["source"] = "system_instructions",
    });
}

return new ValueTask<AIContext>(context.AIContext);
}
}

```

This catches injection attempts that may have been introduced through user-managed data (personas, context entries, etc.) before they reach the LLM.

GovernedMcpGateway

Wraps the AgentGovernance McpGateway to evaluate tool calls against governance policies. It checks whether a tool call should be allowed, denied, or requires approval.

```

public sealed class GovernedMcpGateway
{
    private readonly McpGateway _gateway;

    public GovernedMcpGateway(McpGatewayConfig config, ILogger<GovernedMcpGateway> logger)
    {
        _gateway = new McpGateway(config);
    }

    public McpGatewayDecision Evaluate(string agentId, string toolName, string? payload = null)
    {
        var request = new McpGatewayRequest
        {
            AgentId = agentId,
            ToolName = toolName,
            Payload = payload ?? "",
        };

        var decision = _gateway.ProcessRequest(request);

        if (!decision.Allowed)
            _logger.LogWarning("[GOVERNANCE:MCP] Blocked tool={Tool} agent={Agent}",
                toolName, agentId);

        return decision;
    }
}

```

McpGatewayConfig

```

var mcpConfig = new McpGatewayConfig
{
    BlockOnSuspiciousPayload = true,
    ApprovalRequiredTools = ["run_script", "write_file"],
};

```

Option	Default	Description
BlockOnSuspiciousPayload	true	Block tools with suspicious payloads
ApprovalRequiredTools	["run_script", "write_file"]	Tools that require explicit approval

Governance Policy (YAML)

Policies are defined in `governance-policy.yaml` and evaluated by the `PolicyEngine`. Each rule has a name, scope, action, conditions, and reason.

```
name: AiAgentCanvas-default
description: Default governance policy for AiAgentCanvas

rules:
- name: block-dangerous-tools
  scope: tool_call
  action: deny
  conditions:
  tool_name:
  in: [run_script]
  reason: "Shell execution requires explicit approval"

- name: restrict-file-write-paths
  scope: tool_call
  action: deny
  conditions:
  tool_name:
  equals: write_file
  path:
  matches: "^(/etc|/var|C:\\\\Windows|C:\\\\Program Files)"
  reason: "Writing to system directories is blocked"

- name: block-private-mcp-endpoints
  scope: tool_call
  action: deny
  conditions:
  tool_name:
  equals: connect_mcp_server
  endpoint:
  matches: "(localhost|127\\.0\\.0\\.1|169\\.254\\.|10\\.|172\\.|(1[6-9]|2[0-9]|3[01])\\.|192\\.168\\.)"
  reason: "MCP connections to private addresses are blocked (SSRF protection)"

- name: allow-all-other
  scope: tool_call
  action: allow
  conditions: {}
```

Rule Structure

Field	Description
name	Unique rule identifier
scope	What the rule applies to (tool_call)
action	allow or deny

conditions	Matching criteria: equals, in, matches (regex)
reason	Human-readable explanation logged on match

Rules are evaluated in order. The `ConflictStrategy` of `DenyOverrides` means if any rule denies, the action is blocked regardless of subsequent allow rules.

Configuration

The policy file path is configured in `appsettings.json`:

```
{
  "Security": {
    "PolicyPath": "governance-policy.yaml",
    "RateLimitPerMinute": 30
  }
}
```

ASP.NET Rate Limiting

A fixed-window rate limiter protects the agent endpoint:

```
var rateLimitWindow = configuration.GetValue("Security:RateLimitPerMinute", 30);
services.AddRateLimiter(options =>
{
  options.RejectionStatusCode = StatusCodes.Status429TooManyRequests;
  options.AddFixedWindowLimiter("agent", limiter =>
  {
    limiter.PermitLimit = rateLimitWindow;
    limiter.Window = TimeSpan.FromMinutes(1);
    limiter.QueueLimit = 0;
  });
});
```

When the limit is exceeded, the response is:

```
{"error": "Rate limit exceeded. Try again later."}
```

Security Headers

`UseAiAgentCanvasSecurity()` adds the following headers to every response:

Header	Value	Purpose
X-Content-Type-Options	nosniff	Prevents MIME-type sniffing
X-Frame-Options	DENY	Prevents clickjacking
Referrer-Policy	strict-origin-when-cross-origin	Controls referrer information

Customizing Security

Override defaults by passing configuration callbacks:

```
builder.Services.AddAiAgentCanvasSecurity(builder.Configuration,
configureGovernance: options =>
{
options.EnablePromptInjectionDetection = true;
options.ConflictStrategy = ConflictResolutionStrategy.AllowOverrides;
},
configureMcp: config =>
{
config.ApprovalRequiredTools = ["run_script", "write_file", "delete_file"];
config.BlockOnSuspiciousPayload = true;
});
```

Adding Custom Agents

Custom agents in AI Agent Canvas are lightweight projects that provide specialized system prompts. An agent does not need to implement any interface or reference any framework project -- it can be a pure class library containing only a prompts class. Tools are provided automatically by the `DynamicToolRegistry` and startup-registered tool providers.

How Agents Get Tools

A common question is: how does a custom agent access tools? The answer is that it does not manage tools at all. The Core framework collects all registered `ICollection<AITool>` services and passes them to the `ChatClientAgent`. Your agent's system prompt simply instructs the LLM which tools to use and how to use them. The tools themselves are registered separately (see [Adding Custom MCP Connections](#)).

Step-by-Step Guide

Step 1: Create a Project Folder

Create a new folder under `src/Custom/`:

```
src/Custom/MyAgent/
```

Step 2: Create the .csproj

Create a minimal project file. For a pure prompts agent, no package references are needed:

```
<Project Sdk="Microsoft.NET.Sdk">

  <PropertyGroup>
    <TargetFramework>net9.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
  </PropertyGroup>

</Project>
```

Step 3: Create the Agent Prompts Class

Create a static class with a `GetSystemPrompt()` method that returns the system prompt for your agent:

```
namespace MyAgent;

public static class MyAgentPrompts
{
    public static string GetSystemPrompt() => """
    You are a customer support assistant for Contoso Electronics.

    You have access to the following tools:
    - search_knowledge_base: Search the product knowledge base
    - lookup_order: Look up an order by order number
    """
```

```
- create_ticket: Create a support ticket
```

Guidelines:

```
- Always greet the customer warmly
- Look up order details before making claims about order status
- If you cannot resolve an issue, create a support ticket
- Never share internal pricing or margin information
- Escalate billing disputes to a human agent
""";
}
```

The system prompt should:

- Describe the agent's role and personality
- List the tools it should use (these are registered separately)
- Provide behavioral guidelines and constraints
- Specify what the agent should and should not do

Step 4: Add ProjectReference in AiAgentCanvas.Web.csproj

Open `src/AiAgentCanvas.Web/AiAgentCanvas.Web.csproj` and add a reference to your project:

```
<ItemGroup>
<ProjectReference Include="..\Custom\MyAgent\MyAgent.csproj" />
</ItemGroup>
```

Step 5: Add to AiAgentCanvas.sln

Add the project to the solution under the Custom solution folder:

```
dotnet sln AiAgentCanvas.sln add src/Custom/MyAgent/MyAgent.csproj --solution-folder Custom
```

Step 6: Wire Up in Program.cs

Use your agent's system prompt when configuring `AddAiAgentCanvas()`:

```
using MyAgent;

builder.Services.AddAiAgentCanvas(builder.Configuration, options =>
{
options.AgentName = "CustomerSupport";
options.AgentDescription = "A customer support assistant for Contoso Electronics.";
options.SystemPrompt = MyAgentPrompts.GetSystemPrompt();
});
```

Full Reference: HelloWorldAgent

The `HelloWorldAgent` in `src/Custom/HelloWorldAgent/` is the built-in sample. It has exactly two files.

HelloWorldAgent.csproj


```
<Project Sdk="Microsoft.NET.Sdk">

<PropertyGroup>
<TargetFramework>net9.0</TargetFramework>
<ImplicitUsings>enable</ImplicitUsings>
<Nullable>enable</Nullable>
</PropertyGroup>

</Project>
```

No package references. No project references. Just a net9.0 class library.

HelloWorldAgentPrompts.cs

```
namespace HelloWorldAgent;

public static class HelloWorldAgentPrompts
{
    public static string GetSystemPrompt() => """
    You are a helpful stock market assistant.

    You have access to the following tools:
    - stock_quote: Get the current price and daily change for a ticker
    - stock_technicals: Get technical indicators (RSI, SMA, EMA, MACD)
    - edgar_company_facts: Fetch SEC EDGAR financial data for a company

    When a user asks about a stock, use the appropriate tools to fetch live data,
    then summarize the results in a clear, concise response. Use specific numbers.
    If the user doesn't specify a ticker, ask them which stock they're interested in.
    """;
}
```

The tools referenced in the prompt (stock_quote, stock_technicals, edgar_company_facts) are provided by the MCP.MarketData custom project. They are registered separately via builder.Services.AddMarketDataTools() in Program.cs.

Switching Between Agents

To switch between agents, change the SystemPrompt in the AddAiAgentCanvas() call in Program.cs. You can also use the Personas agent data domain to switch prompts at runtime without restarting the server -- see [Agent Data](#).

Agents with Custom Tools

If your agent needs custom tools (data connections, API integrations), create them as a separate Custom project following the [Adding Custom MCP Connections](#) guide. The separation is intentional: agent prompts define behavior, while tool providers define capabilities.

However, if you prefer to keep tools and prompts in the same project, you can. Just add the Microsoft.Extensions.AI package reference and follow the tool provider pattern:

```
// MyAgent/MyAgentToolProvider.cs
using Microsoft.Extensions.AI;

namespace MyAgent;
```

```

public sealed class MyAgentToolProvider
{
    public IReadOnlyList<AITool> GetTools()
    {
        return
        [
            AIFunctionFactory.Create(SearchKnowledgeBase, "search_knowledge_base",
            "Search the product knowledge base"),
        ];
    }

    private string SearchKnowledgeBase(string query)
    {
        // Your implementation here
        return "...";
    }
}

// MyAgent/MyAgentServiceExtensions.cs
using Microsoft.Extensions.AI;
using Microsoft.Extensions.DependencyInjection;

namespace MyAgent;

public static class MyAgentServiceExtensions
{
    public static IServiceCollection AddMyAgent(this IServiceCollection services)
    {
        services.AddSingleton<MyAgentToolProvider>();
        services.AddSingleton<IReadOnlyList<AITool>>(sp =>
            sp.GetRequiredService<MyAgentToolProvider>().GetTools());
        return services;
    }
}

```

Then in Program.cs:

```
builder.Services.AddMyAgent();
```

Testing Your Agent

1. Build the solution: `dotnet build AiAgentCanvas.sln`
2. Run the backend: `cd src/AiAgentCanvas.Web && dotnet run`
3. Run the frontend: `cd frontend && npm run dev`
4. Open the CopilotKit UI and interact with your agent
5. Check the console logs for tool calls and governance events

Adding Custom MCP Connections

Custom MCP connections in AI Agent Canvas are projects that provide tools (data connections, API integrations) to the agent. Each connection is a class library that defines tools using `AIFunctionFactory.Create()` and registers them as `IReadOnlyList<AITool>` services.

The term "MCP connection" here refers to a local tool provider project (not an external MCP server -- for connecting to remote MCP servers at runtime, see [Skills and MCP](#)).

How Tool Registration Works

The Core framework collects all `IReadOnlyList<AITool>` singleton services at startup and passes them to the `ChatClientAgent`:

```
// Inside AddAiAgentCanvas() in Core:
var tools = sp.GetServices<IReadOnlyList<AITool>>().SelectMany(t => t).ToList();
var agentOptions = new ChatClientAgentOptions
{
    ChatOptions = new ChatOptions { Tools = tools },
};
```

Your custom project just needs to register its tools as `IReadOnlyList<AITool>` and they automatically become available to the LLM.

Step-by-Step Guide

Step 1: Create the Project

Create a new folder under `src/Custom/` and a `.csproj` with the `Microsoft.Extensions.AI` package:

```
src/Custom/MCP.Weather/

<Project Sdk="Microsoft.NET.Sdk">

  <PropertyGroup>
    <TargetFramework>net9.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
  </PropertyGroup>

  <ItemGroup>
    <PackageReference Include="Microsoft.Extensions.AI" Version="10.7.0" />
  </ItemGroup>

</Project>
```

Add `Microsoft.Extensions.Http.Resilience` if you need HTTP clients with retry policies.

Step 2: Create a ToolProvider Class

Define your tools using `AIFunctionFactory.Create()`. Each tool is a method decorated with `[Description]` attributes:

```
using System.ComponentModel;
using System.Net.Http.Json;
using System.Text.Json;
using Microsoft.Extensions.AI;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.Logging;

namespace MCP.Weather;

public sealed class WeatherToolProvider
{
    private readonly IHttpClientFactory _httpClientFactory;
    private readonly string _apiKey;
    private readonly ILogger<WeatherToolProvider> _logger;

    public WeatherToolProvider(
        IHttpClientFactory httpClientFactory,
        IConfiguration configuration,
        ILogger<WeatherToolProvider> logger)
    {
        _httpClientFactory = httpClientFactory;
        _apiKey = configuration["Weather:ApiKey"] ?? "";
        _logger = logger;
    }

    public IReadOnlyList<AITool> GetTools()
    {
        return
        [
            AIFunctionFactory.Create(GetCurrentWeather, "get_current_weather",
                "Get current weather conditions for a city"),
            AIFunctionFactory.Create(GetForecast, "get_weather_forecast",
                "Get a 5-day weather forecast for a city"),
        ];
    }

    [Description("Get current weather conditions for a city")]
    private async Task<string> GetCurrentWeather(
        [Description("City name (e.g. 'Seattle', 'London')")] string city,
        CancellationToken ct)
    {
        try
        {
            var client = _httpClientFactory.CreateClient("Weather");
            var url = $"https://api.weather.example.com/current?q={city}&key={_apiKey}";
            var result = await client.GetStringAsync(url, ct);

            _logger.LogInformation("Weather fetched for {City}", city);
            return result;
        }
        catch (HttpRequestException ex)
        {
            _logger.LogWarning(ex, "Weather fetch failed for {City}", city);
            return JsonSerializer.Serialize(new { error = $"Failed: {ex.Message}" });
        }
    }

    [Description("Get a 5-day weather forecast for a city")]
```

```
private async Task<string> GetForecast(
    [Description("City name")] string city,
    [Description("Number of days (1-5)")] int days,
    CancellationToken ct)
{
    // Implementation similar to above
    return "...";
}
}
```

Key points:

- The constructor receives dependencies from DI (IHttpClientFactory, IConfiguration, ILogger)
- GetTools() returns the tool list using AIFunctionFactory.Create()
- Each tool method uses [Description] attributes so the LLM understands parameters
- Tools return JSON strings (the LLM parses the response)
- Always handle errors gracefully and return error JSON instead of throwing

Step 3: Create ServiceExtensions

Create an extension method that registers the HttpClient, the tool provider, and the tools:

```
using Microsoft.Extensions.AI;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Http.Resilience;

namespace MCP.Weather;

public static class WeatherServiceExtensions
{
    public static IServiceCollection AddWeatherTools(this IServiceCollection services)
    {
        services.AddHttpClient("Weather", client =>
        {
            client.Timeout = TimeSpan.FromSeconds(15);
        })
        .AddStandardResilienceHandler();

        services.AddSingleton<WeatherToolProvider>();
        services.AddSingleton<IReadOnlyList<AITool>>(sp =>
            sp.GetRequiredService<WeatherToolProvider>().GetTools());

        return services;
    }
}
```

The critical line is the `IReadOnlyList<AITool>` registration. This is how the Core framework discovers your tools.

Step 4: Wire Up in Program.cs

Add the project reference and call your extension method:

```
<!-- In AiAgentCanvas.Web.csproj -->
<ItemGroup>
```

```
<ProjectReference Include="..\Custom\MCP.Weather\MCP.Weather.csproj" />
</ItemGroup>
```

```
// In Program.cs
using MCP.Weather;

builder.Services.AddWeatherTools();
```

Add the project to the solution:

```
dotnet sln AiAgentCanvas.sln add src/Custom/MCP.Weather/MCP.Weather.csproj
--solution-folder Custom
```

Full Reference: MCP.MarketData

The `MCP.MarketData` project in `src/Custom/MCP.MarketData/` is the built-in reference implementation. It provides three tools for stock market data.

MarketDataToolProvider.cs

The tool provider defines three tools:

```
public IReadOnlyList<AITool> GetTools()
{
    return
    [
        AIFunctionFactory.Create(EdgarCompanyFactsAsync, "edgar_company_facts",
            "Fetch SEC EDGAR financial data for a company by ticker symbol"),
        AIFunctionFactory.Create(StockQuoteAsync, "stock_quote",
            "Get current stock price and daily change from Alpha Vantage"),
        AIFunctionFactory.Create(StockTechnicalAsync, "stock_technicals",
            "Get technical indicators (RSI, SMA, EMA, MACD) from Alpha Vantage"),
    ];
}
```

Each tool:

- Takes typed parameters with [Description] attributes
- Uses `IHttpClientFactory` to make HTTP requests
- Returns JSON strings with structured data or error information
- Logs timing and errors via `ILogger`
- Accepts `CancellationToken` for proper cancellation

MarketDataServiceExtensions.cs

```
public static IServiceCollection AddMarketDataTools(this IServiceCollection services)
{
    services.AddHttpClient("SEC", client =>
    {
        client.Timeout = TimeSpan.FromSeconds(30);
        client.DefaultRequestHeaders.UserAgent.ParseAdd(
            "AiAgentCanvas/1.0 (contact@example.com)");
    })
}
```

```

.AddStandardResilienceHandler();

services.AddHttpClient("AlphaVantage", client =>
{
client.Timeout = TimeSpan.FromSeconds(15);
})
.AddStandardResilienceHandler();

services.AddSingleton<MarketDataToolProvider>();
services.AddSingleton<IReadOnlyList<AITool>>(sp =>
sp.GetRequiredService<MarketDataToolProvider>().GetTools());

return services;
}

```

Notable patterns:

- Named HttpClients for different API backends
- `AddStandardResilienceHandler()` for automatic retry and circuit-breaking
- Custom User-Agent header for the SEC EDGAR API (which requires identification)
- Separate timeout values per API

Testing Tools

Once registered, your tools are automatically discovered by the agent. To test:

1. Build: `dotnet build AiAgentCanvas.sln`
2. Run: `cd src/AiAgentCanvas.Web && dotnet run`
3. Open the CopilotKit frontend
4. Ask the agent to use your tool (e.g., "What's the weather in Seattle?")
5. Check the backend console for tool call logs

The LLM discovers tools from their names and descriptions. Make sure your tool names are descriptive and your `[Description]` attributes clearly explain what each parameter expects.

Tool Design Guidelines

1. **Return JSON strings** -- The LLM parses your tool's return value. Always return structured JSON.
2. **Handle errors gracefully** -- Return `{ "error": "..."}` instead of throwing exceptions.
3. **Use Cancellation Token** -- Accept and pass through the cancellation token for proper request cancellation.
4. **Log appropriately** -- Use `ILogger` for timing, errors, and debugging.
5. **Keep tools focused** -- One tool per API operation. Let the LLM compose multiple tool calls.
6. **Descriptive names** -- Use snake_case names that clearly describe the action (e.g., `get_current_weather`, not `weather`).

7. **Document parameters** -- Every parameter needs a [Description] attribute with examples.

Tools Without External APIs

Not every tool needs HTTP calls. You can create tools that do local computation, file operations, or database queries:

```
public IReadOnlyList<AITool> GetTools()
{
    return
    [
        AIFunctionFactory.Create(CalculateCompoundInterest, "calculate_compound_interest",
            "Calculate compound interest over a period"),
    ];
}

[Description("Calculate compound interest")]
private string CalculateCompoundInterest(
    [Description("Principal amount")] decimal principal,
    [Description("Annual interest rate (e.g. 0.05 for 5%)")] decimal rate,
    [Description("Number of years")] int years)
{
    var result = principal * (decimal)Math.Pow((double)(1 + rate), years);
    return JsonSerializer.Serialize(new { principal, rate, years, futureValue = result });
}
```

These tools follow the same registration pattern and are automatically available to the agent.